

Importing library

Nabin Gangtan Lama (st125985)

- github link: https://github.com/lamanabin2046/A3_CAR_PRICE_PREDICTION

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import sklearn
warnings.filterwarnings("ignore", category=UserWarning)
```

```
import matplotlib
np.__version__, pd.__version__, sns.__version__,
matplotlib.__version__, sklearn.__version__

('1.26.4', '1.5.3', '0.12.2', '3.7.2', '1.7.1')
```

```
# Loading the Cars.csv Data sets
df= pd.read_csv('Cars.csv')
```

```
# Displaying first five samples
df.head(5)
```

	fuel	brand	year	selling_price	km_driven
0	Diesel	Maruti Swift Dzire VDI	2014	450000	145500
1	Diesel	Skoda Rapid 1.5 TDI Ambition	2014	370000	120000
2	Petrol	Honda City 2017-2020 EXi	2006	158000	140000
3	Diesel	Hyundai i20 Sportz Diesel	2010	225000	127000
4	Petrol	Maruti Swift VXi BSIII	2007	130000	120000

	seller_type	transmission	owner	mileage	engine
0	Individual	Manual	First Owner	23.4 kmpl	1248 CC 74 bhp
1	Individual	Manual	Second Owner	21.14 kmpl	1498 CC 103.52 bhp
2	Individual	Manual	Third Owner	17.7 kmpl	1497 CC 78 bhp
3	Individual	Manual	First Owner	23.0 kmpl	1396 CC 90 bhp
4	Individual	Manual	First Owner	16.1 kmpl	1298 CC 88.2 bhp

	torque	seats
0	190Nm@ 2000rpm	5.0
1	250Nm@ 1500-2500rpm	5.0
2	12.7@ 2,700(kgm@ rpm)	5.0
3	22.4 kgm at 1750-2750rpm	5.0
4	11.5@ 4,500(kgm@ rpm)	5.0

Checking the shape that number of sample size and features
df.shape

(8128, 13)

Checking datasets
df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8128 entries, 0 to 8127
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   brand                  8128 non-null   object
1   year                   8128 non-null   int64
2   selling_price          8128 non-null   int64
3   km_driven              8128 non-null   int64
4   fuel                   8128 non-null   object
5   seller_type            8128 non-null   object
6   transmission           8128 non-null   object
7   owner                  8128 non-null   object
8   mileage                7907 non-null   object
9   engine                 7907 non-null   object
10  max_power              7913 non-null   object
11  torque                 7906 non-null   object
12  seats                  7907 non-null   float64
dtypes: float64(1), int64(3), object(9)
memory usage: 825.6+ KB
```

Checking the nan values
df.isnull().sum()

brand	0
year	0
selling_price	0
km_driven	0
fuel	0
seller_type	0
transmission	0
owner	0
mileage	221
engine	221
max_power	215

```
torque          222
seats           221
dtype: int64
```

```
# Column owner has different categories, so before mapping, checking the categories in owner column
```

```
df['owner'].unique()
```

```
array(['First Owner', 'Second Owner', 'Third Owner',
       'Fourth & Above Owner', 'Test Drive Car'], dtype=object)
```

```
# creating a dictionary mapping and mapping the owner column.
```

```
mapping = {'First Owner':1, 'Second Owner':2, 'Third Owner':3, 'Fourth & Above Owner':4, 'Test Drive Car':5}
```

```
df['owner'] = df['owner'].map(mapping)
```

```
# Verifying the owner category
```

```
df['owner'].value_counts()
```

```
1    5289
```

```
2    2105
```

```
3     555
```

```
4     174
```

```
5         5
```

```
Name: owner, dtype: int64
```

```
# for feature fuel, remove all rows with CNG and LPG. Before that lets check the categories in the feature fuel
```

```
df['fuel'].unique()
```

```
array(['Diesel', 'Petrol', 'LPG', 'CNG'], dtype=object)
```

```
# Selecting the rows of feature fuel that does not contain CNG and LPG
```

```
df = df[~df['fuel'].isin(['CNG', 'LPG'])]
```

```
df['fuel'].unique()
```

```
array(['Diesel', 'Petrol'], dtype=object)
```

```
# Removing the kmpl from the feature mileage and converting the object data types to the float.
```

```
df['mileage'] = df['mileage'].str.split().str[0].astype(float)
```

```
# Removing the CC from the feature engine and converting the object data types to the float.
```

```
df['engine'] = df['engine'].str.split().str[0].astype(float)
```

```
df['engine'].head()
```

```
0    1248.0
```

```
1    1498.0
```

```
2    1497.0
```

```
3    1396.0
```

```
4    1298.0
Name: engine, dtype: float64
```

```
# Removing the CC fromt the feature engine and converting the object data types to the float.
```

```
df['max_power']= df['max_power'].str.split().str[0].astype(float)
df['max_power'].head()
```

```
0    74.00
1   103.52
2    78.00
3    90.00
4    88.20
Name: max_power, dtype: float64
```

```
# As mentioned brand, lets change the feature name to brand
df =df.rename(columns={'name':'brand'})
```

```
# Taking only the first word of the brand
df['brand']=df['brand'].str.split().str[0]
```

```
# verifying the df
df.head()
```

	brand	year	selling_price	km_driven	fuel	seller_type
transmission \						
0	Maruti	2014	450000	145500	Diesel	Individual
Manual						
1	Skoda	2014	370000	120000	Diesel	Individual
Manual						
2	Honda	2006	158000	140000	Petrol	Individual
Manual						
3	Hyundai	2010	225000	127000	Diesel	Individual
Manual						
4	Maruti	2007	130000	120000	Petrol	Individual
Manual						

	owner	mileage	engine	max_power	torque	seats
0	1	23.40	1248.0	74.00	190Nm@ 2000rpm	5.0
1	2	21.14	1498.0	103.52	250Nm@ 1500-2500rpm	5.0
2	3	17.70	1497.0	78.00	12.7@ 2,700(kgm@ rpm)	5.0
3	1	23.00	1396.0	90.00	22.4 kgm at 1750-2750rpm	5.0
4	1	16.10	1298.0	88.20	11.5@ 4,500(kgm@ rpm)	5.0

```
# Dropping the feature torque because it is no longer needed
```

```
df = df.drop(columns='torque')
```

```
# verifying the all torque removed or not
```

```
df.head()
```

	brand	year	selling_price	km_driven	fuel	seller_type
0	Maruti	2014	450000	145500	Diesel	Individual
1	Skoda	2014	370000	120000	Diesel	Individual
2	Honda	2006	158000	140000	Petrol	Individual
3	Hyundai	2010	225000	127000	Diesel	Individual
4	Maruti	2007	130000	120000	Petrol	Individual

	owner	mileage	engine	max_power	seats
0	1	23.40	1248.0	74.00	5.0
1	2	21.14	1498.0	103.52	5.0
2	3	17.70	1497.0	78.00	5.0
3	1	23.00	1396.0	90.00	5.0
4	1	16.10	1298.0	88.20	5.0

```
# removing Test Drive Cars from the which was mapped to 5 in the owner feature
```

```
df =df[df['owner'] !=5]
```

```
# verifying Onwer
```

```
df['owner'].unique()
```

```
array([1, 2, 3, 4])
```

```
df.head()
```

	brand	year	selling_price	km_driven	fuel	seller_type
0	Maruti	2014	450000	145500	Diesel	Individual
1	Skoda	2014	370000	120000	Diesel	Individual
2	Honda	2006	158000	140000	Petrol	Individual
3	Hyundai	2010	225000	127000	Diesel	Individual
4	Maruti	2007	130000	120000	Petrol	Individual

	owner	mileage	engine	max_power	seats
--	-------	---------	--------	-----------	-------

0	1	23.40	1248.0	74.00	5.0
1	2	21.14	1498.0	103.52	5.0
2	3	17.70	1497.0	78.00	5.0
3	1	23.00	1396.0	90.00	5.0
4	1	16.10	1298.0	88.20	5.0

```
df.to_csv('Cars_Price.csv', index=False)
```

```
# Another way of checking the all Column along with its data types
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 8028 entries, 0 to 8127
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   brand                  8028 non-null   object
1   year                   8028 non-null   int64
2   selling_price          8028 non-null   int64
3   km_driven               8028 non-null   int64
4   fuel                   8028 non-null   object
5   seller_type            8028 non-null   object
6   transmission           8028 non-null   object
7   owner                  8028 non-null   int64
8   mileage                7814 non-null   float64
9   engine                 7814 non-null   float64
10  max_power              7820 non-null   float64
11  seats                  7814 non-null   float64
dtypes: float64(4), int64(4), object(4)
memory usage: 815.3+ KB
```

```
## Univariate Analysis
```

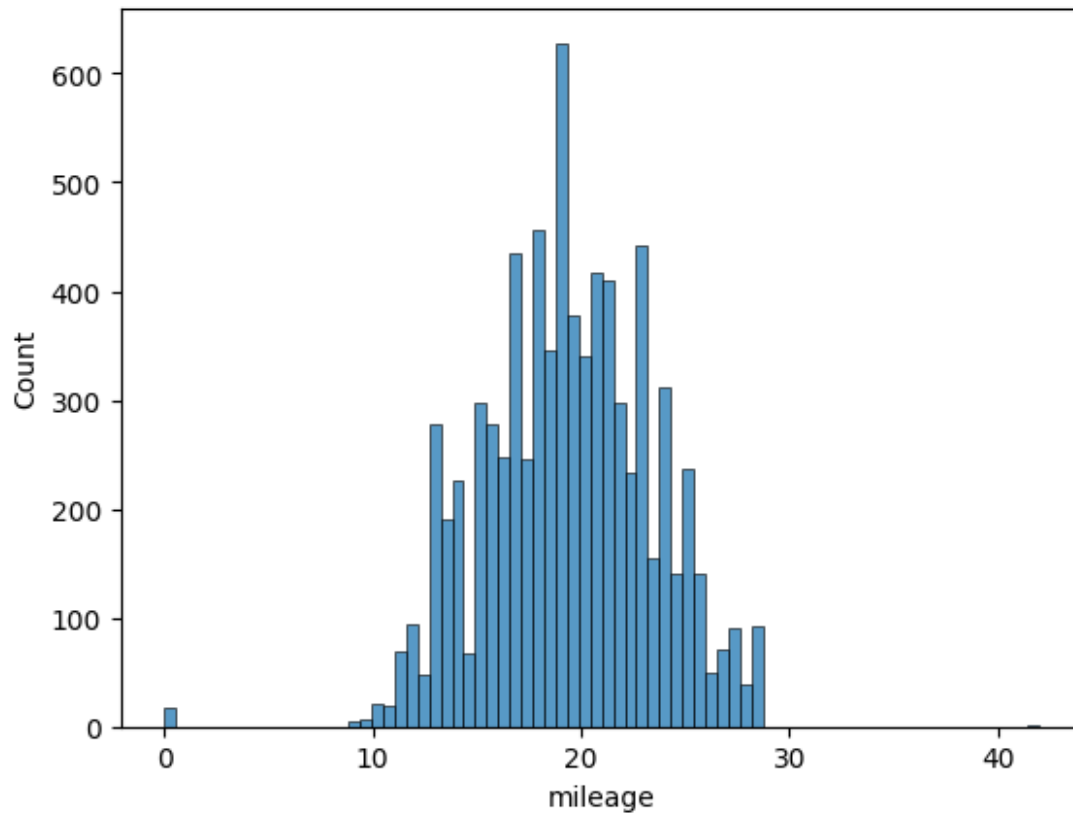
```
num_feature = df.select_dtypes(include=['int64', 'float64'])
```

```
cat_feature = df.select_dtypes(exclude=['int64', 'float64'])
```

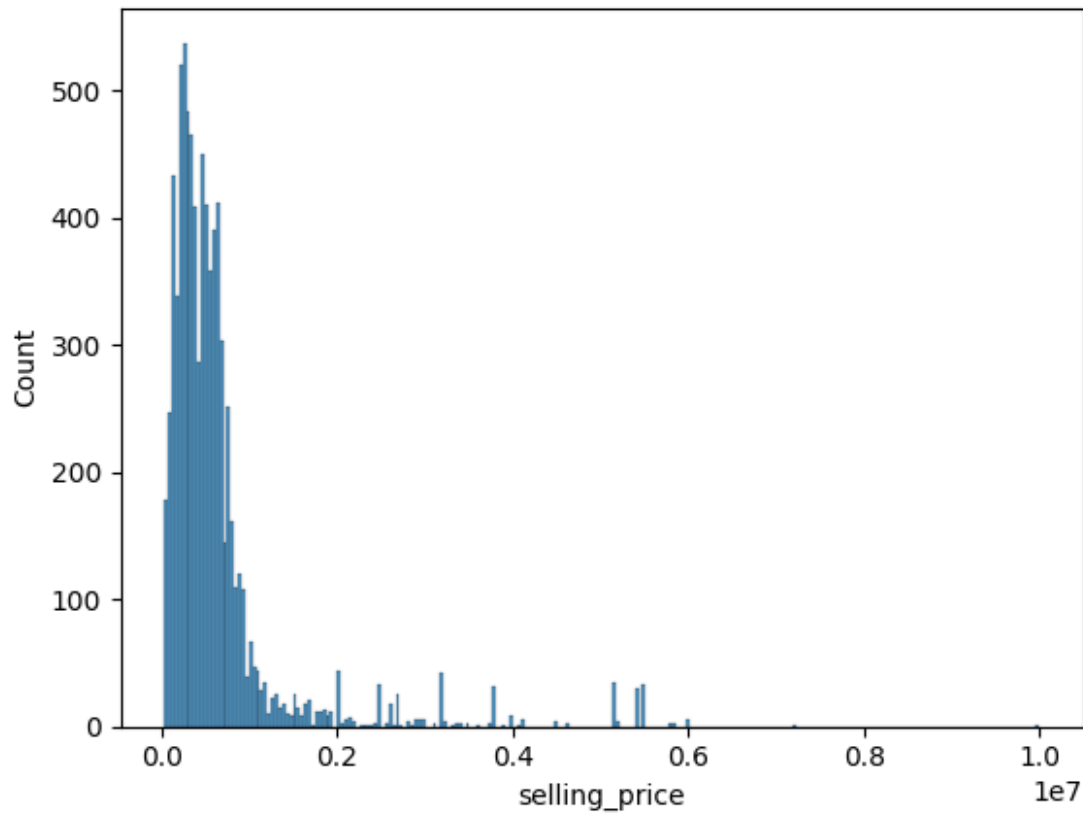
```
# Performing Univariate Analysis
```

```
sns.histplot(data= df, x= 'mileage')
```

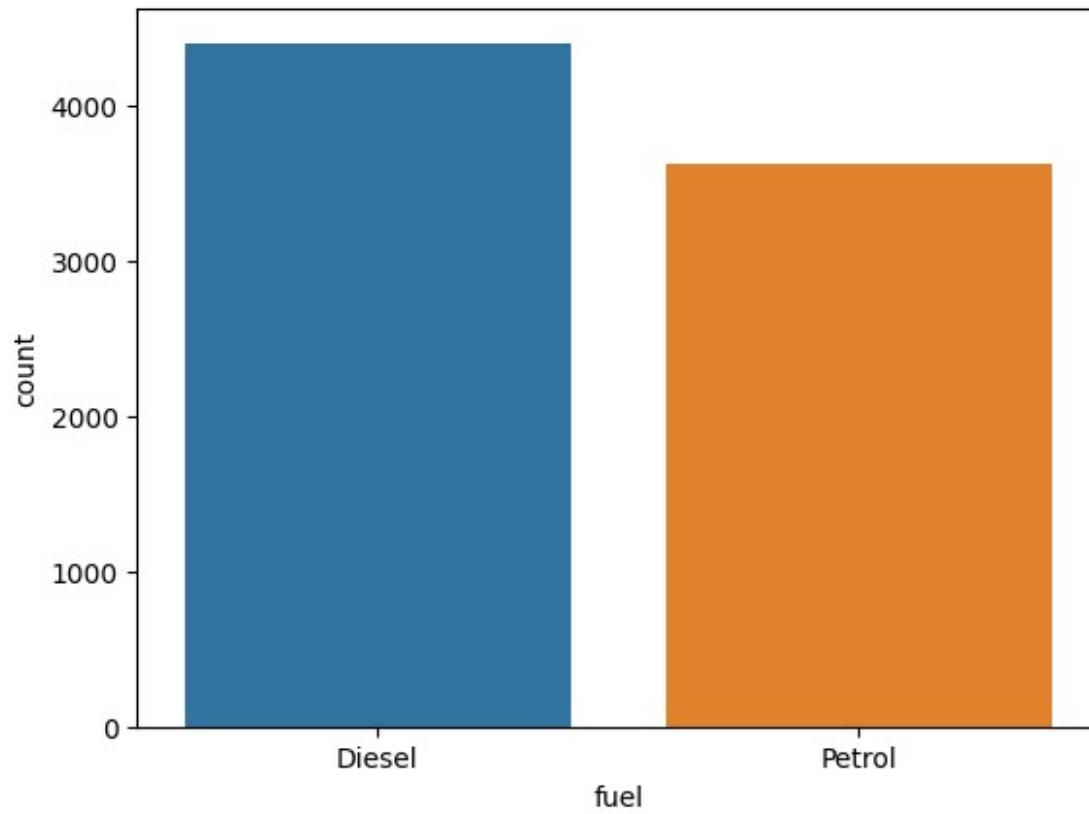
```
<Axes: xlabel='mileage', ylabel='Count'>
```



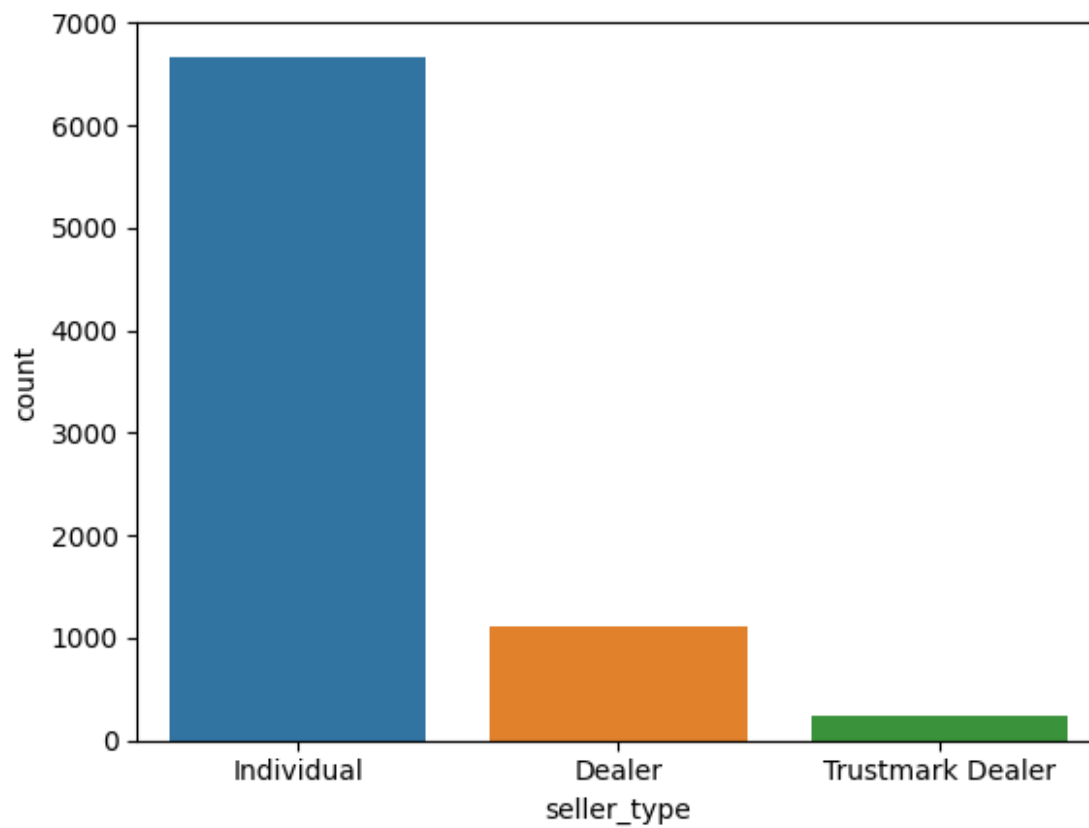
```
# Histogram of Selling Price
sns.histplot(data= df, x= 'selling_price')
<Axes: xlabel='selling_price', ylabel='Count'>
```



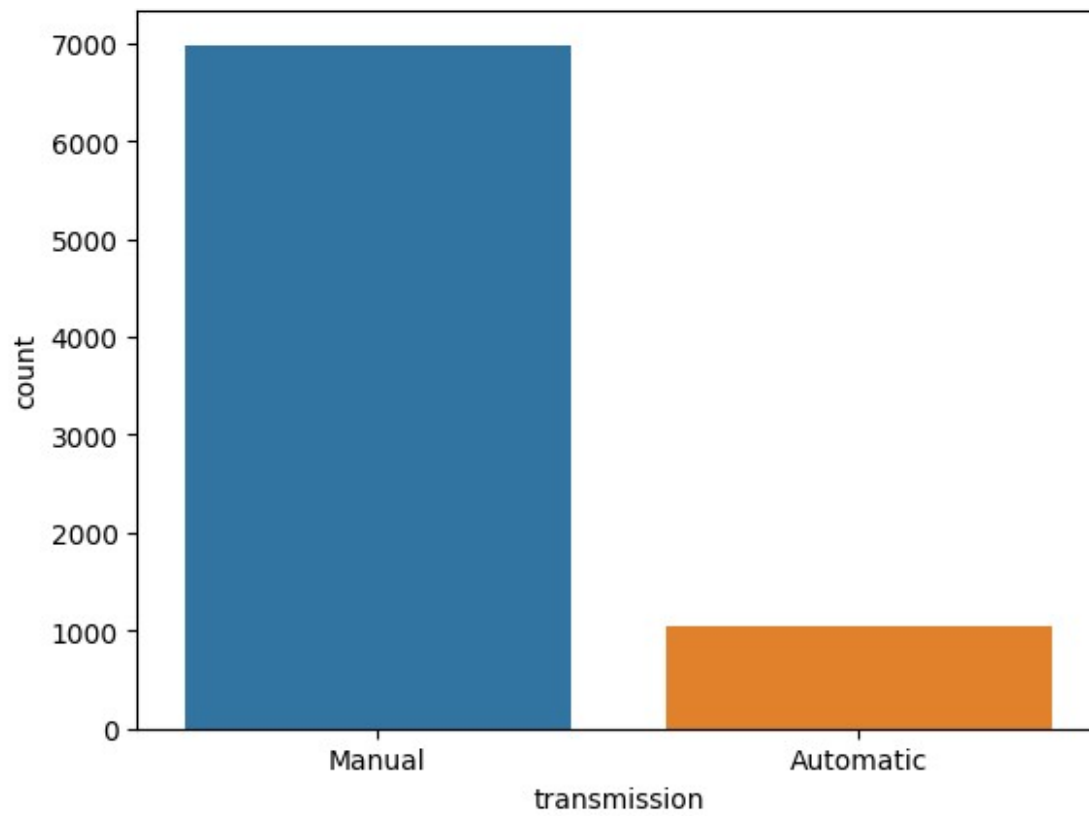
```
# Countplot of Fuel  
sns.countplot(data= df, x= 'fuel')  
<Axes: xlabel='fuel', ylabel='count'>
```

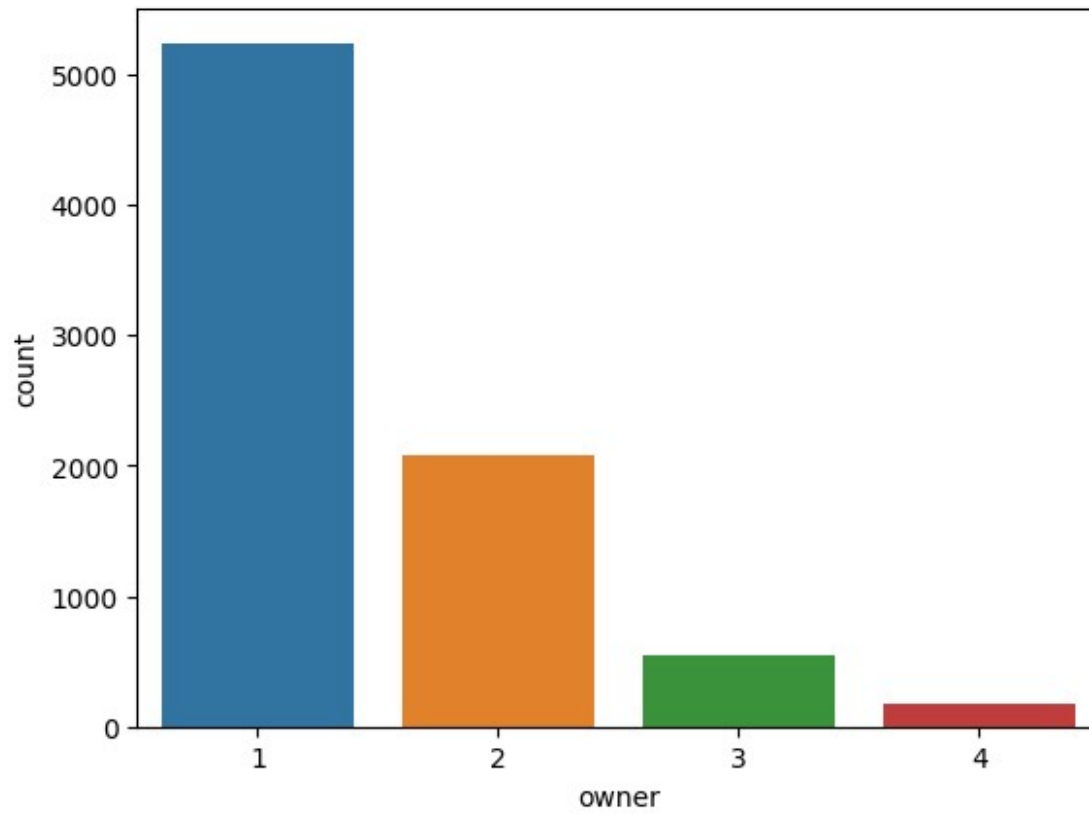
```
# graph of Selling type  
sns.countplot(data= df, x= 'seller_type')  
<Axes: xlabel='seller_type', ylabel='count'>
```



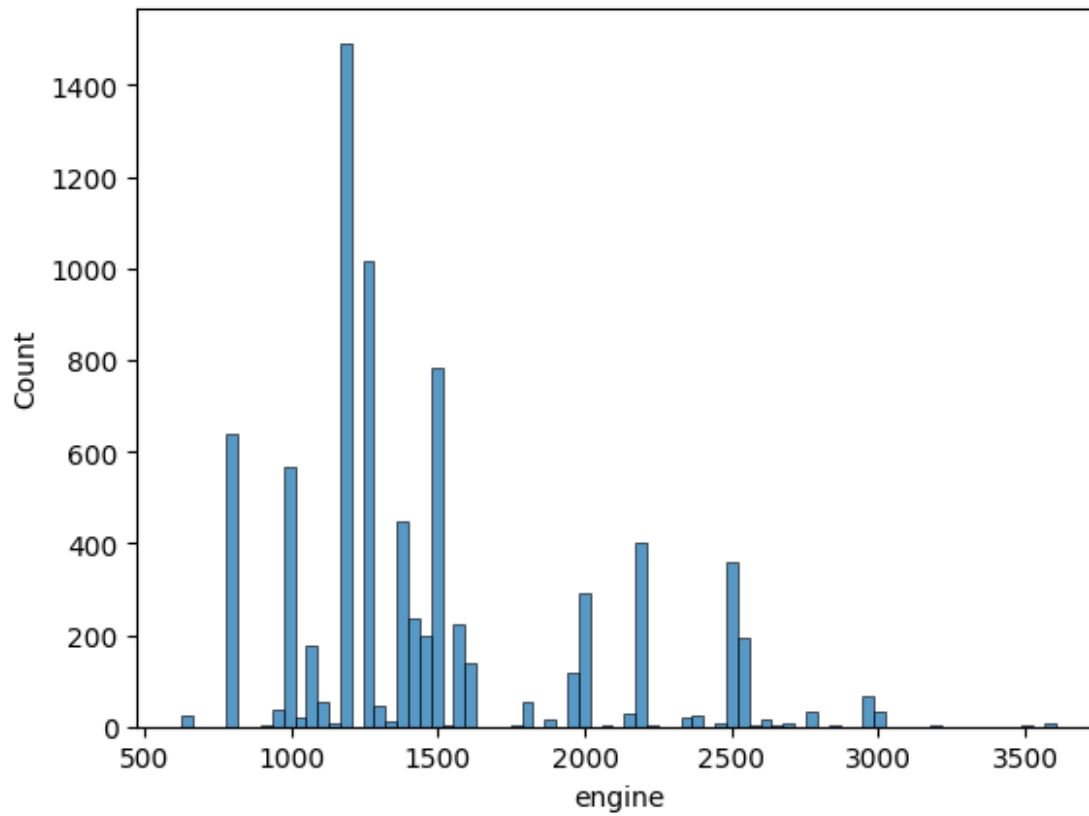
```
sns.countplot(data= df, x= 'transmission')  
<Axes: xlabel='transmission', ylabel='count'>
```



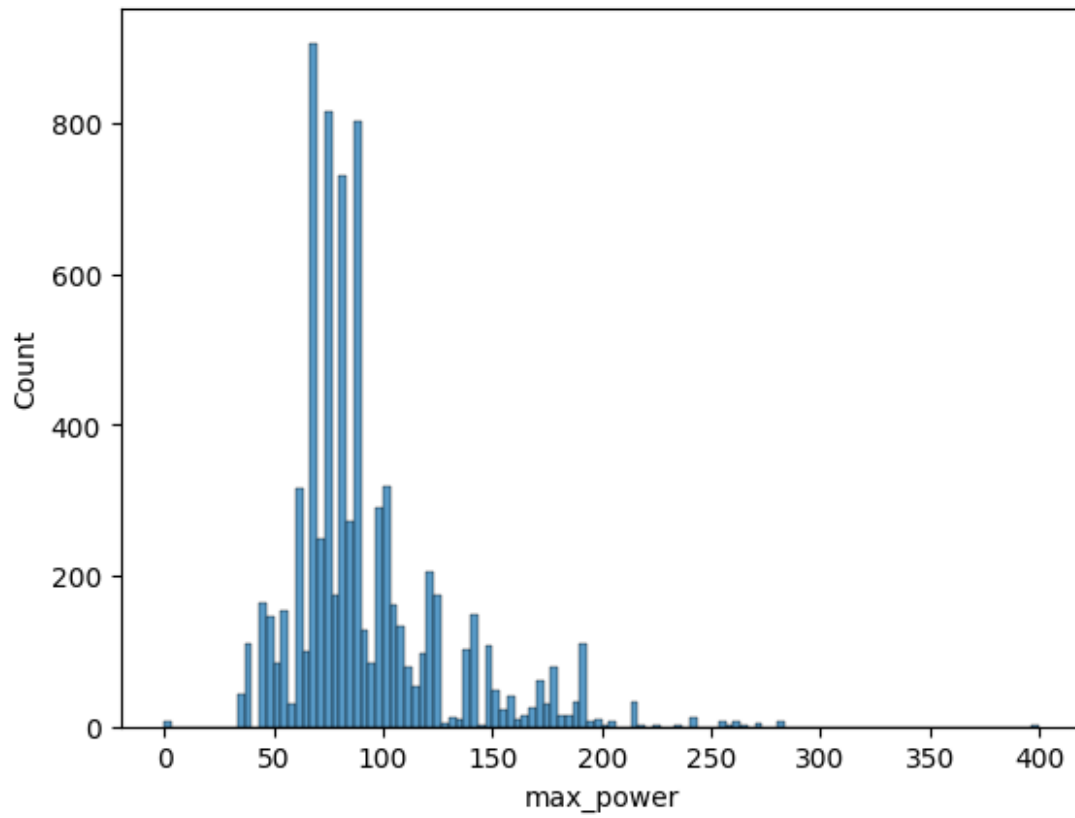
```
sns.countplot(data= df, x='owner')  
<Axes: xlabel='owner', ylabel='count'>
```



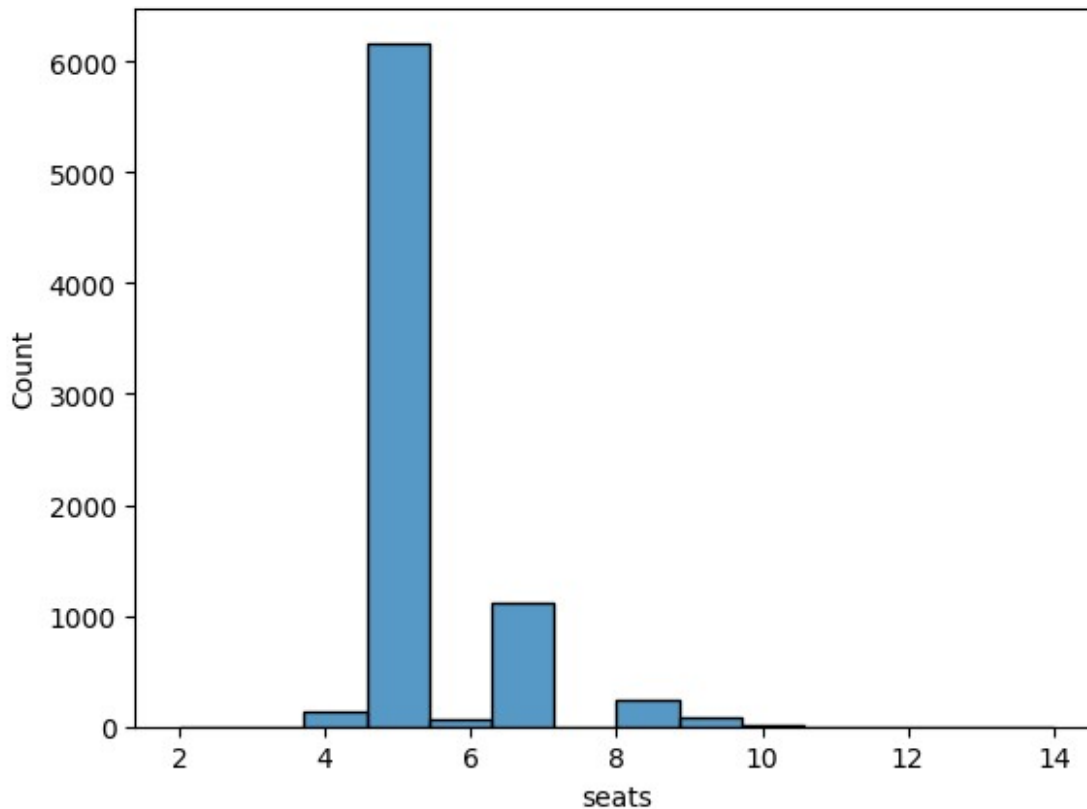
```
sns.histplot(data= df, x= 'engine')  
<Axes: xlabel='engine', ylabel='Count'>
```



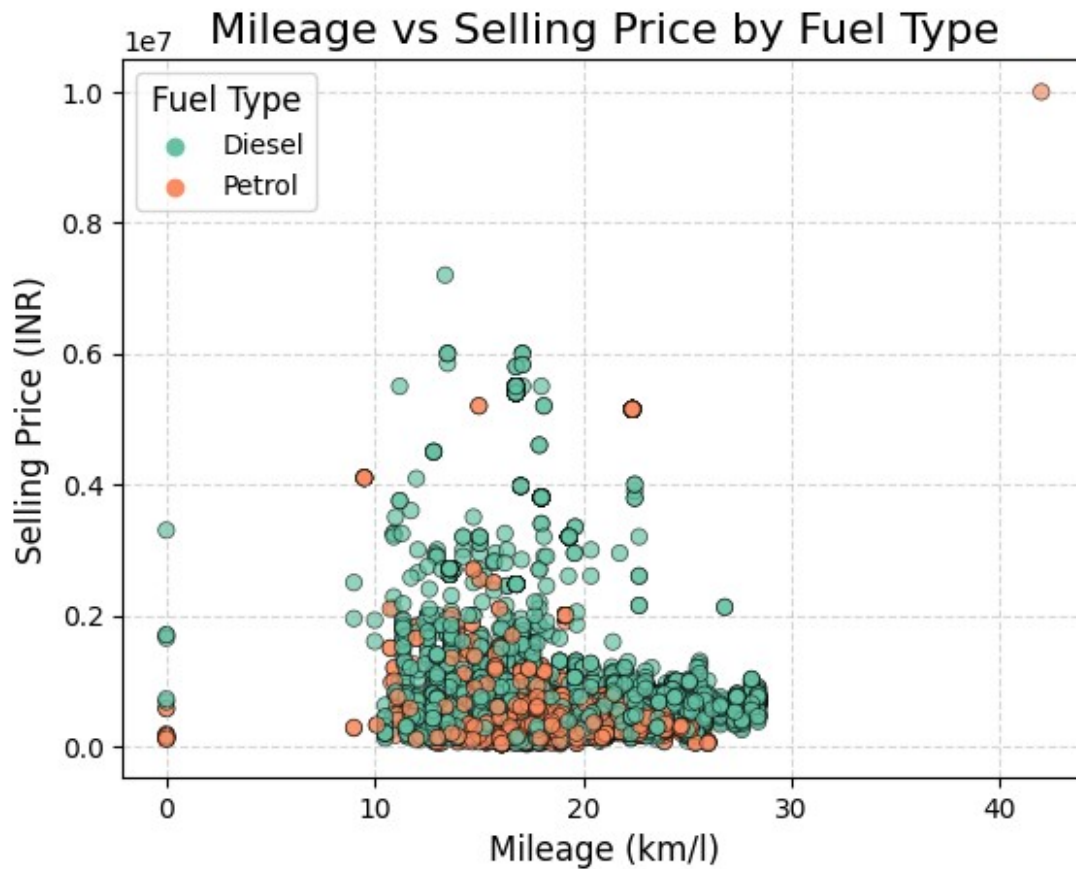
```
sns.histplot(data= df, x= 'max_power')  
<Axes: xlabel='max_power', ylabel='Count'>
```



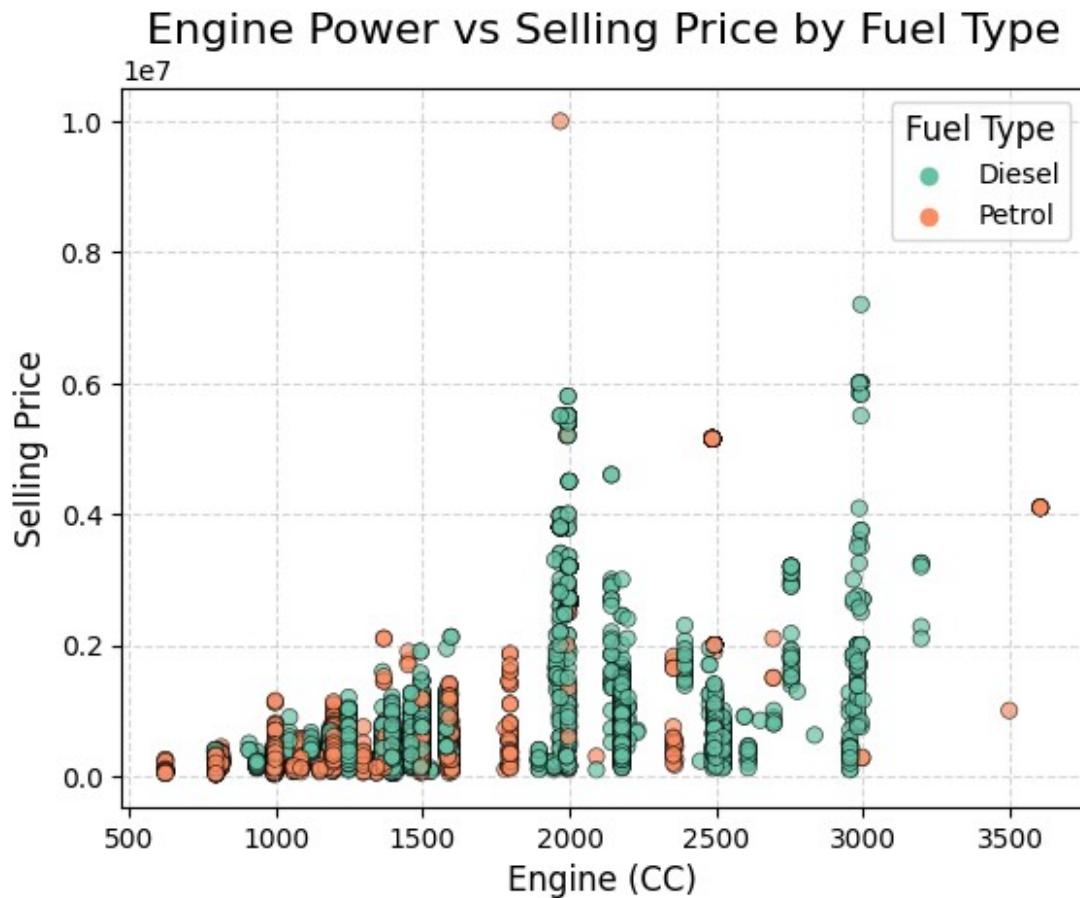
```
sns.histplot(data= df, x= 'seats')  
<Axes: xlabel='seats', ylabel='Count'>
```



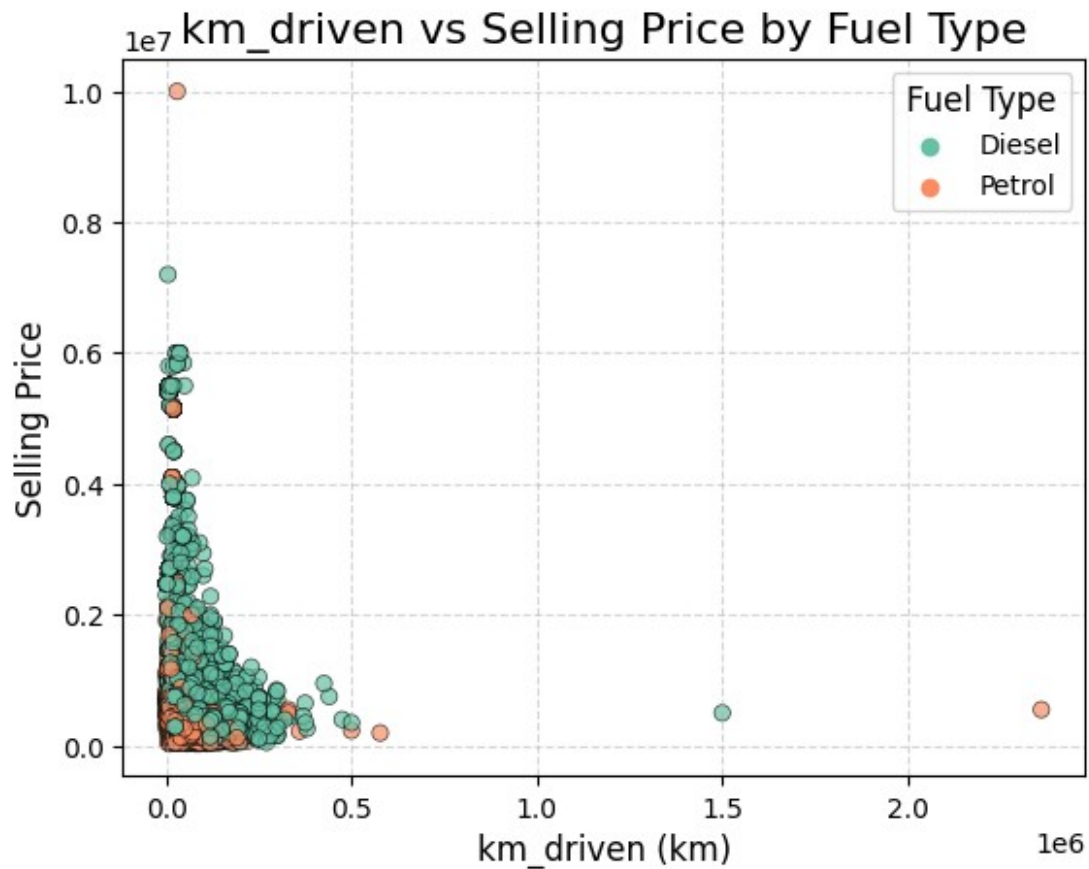
```
# sns.scatterplot(data = df,x='mileage', y= df['selling_price'],
hue='fuel')
sns.scatterplot(data=df, x='mileage', y='selling_price', hue='fuel',
sizes=(10, 20), palette='Set2',alpha=0.7,
edgecolor='k',linewidth=0.5,marker='o')
plt.title("Mileage vs Selling Price by Fuel Type", fontsize=16)
plt.xlabel("Mileage (km/l)", fontsize=12)
plt.ylabel("Selling Price (INR)", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(title='Fuel Type', fontsize=10, title_fontsize=12)
plt.show()
```



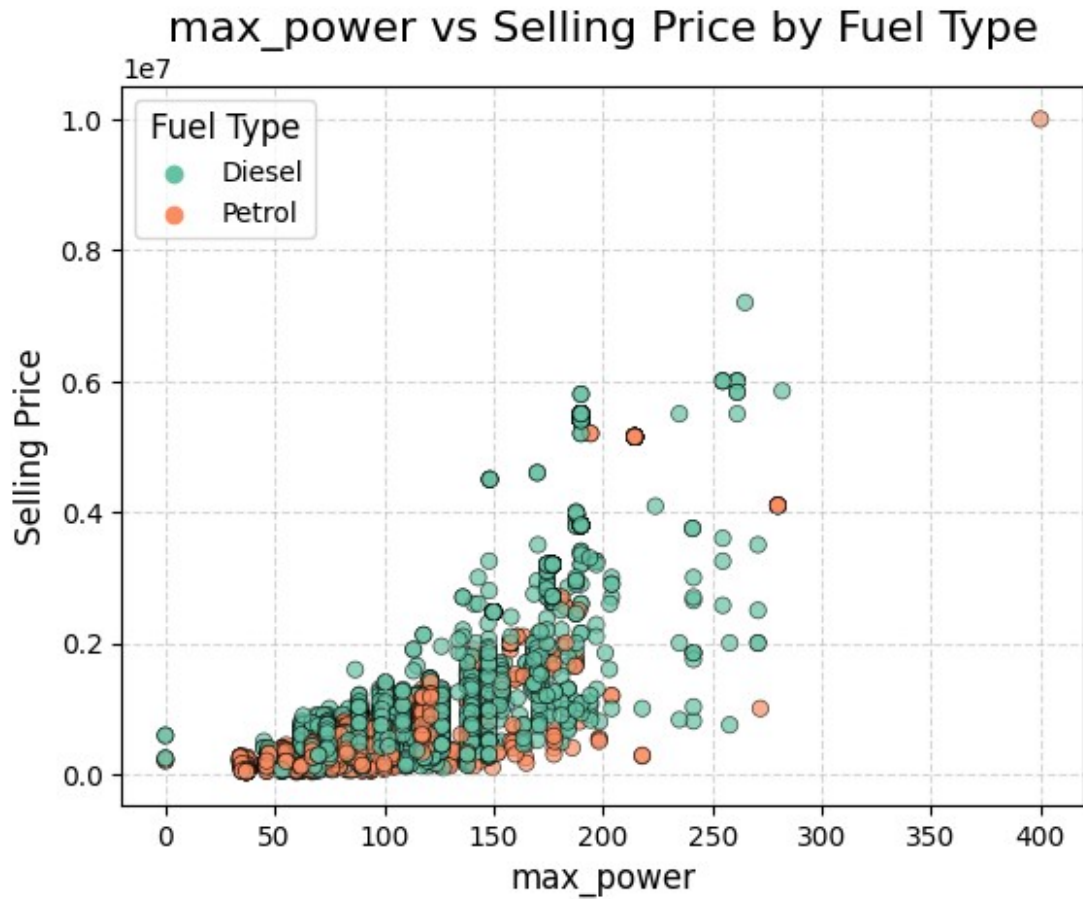
```
sns.scatterplot(data=df, x='engine', y='selling_price', hue='fuel',
                sizes=(10, 20), palette='Set2', alpha=0.7,
                edgecolor='k', linewidth=0.5, marker='o')
plt.title("Engine Power vs Selling Price by Fuel Type", fontsize=16)
plt.xlabel("Engine (CC)", fontsize=12)
plt.ylabel("Selling Price", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(title='Fuel Type', fontsize=10, title_fontsize=12)
plt.show()
```

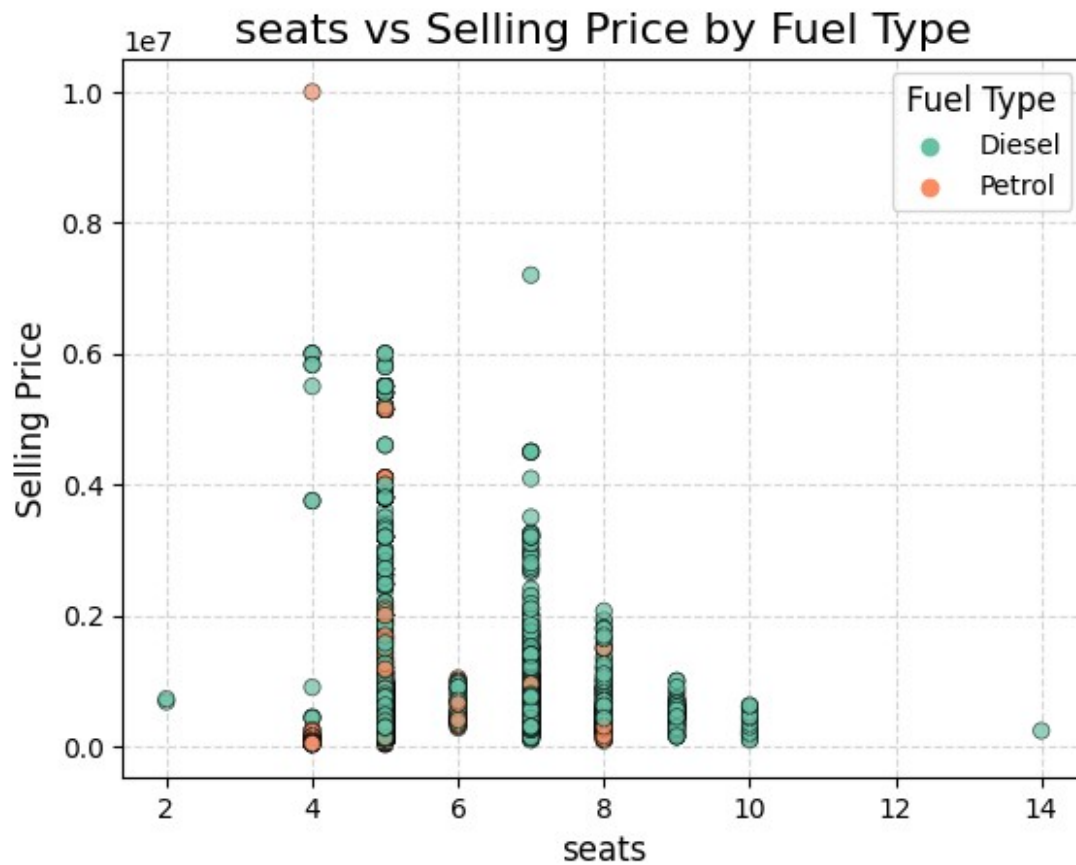
```
sns.scatterplot(data=df, x='km_driven', y='selling_price', hue='fuel',
                sizes=(10, 20), palette='Set2', alpha=0.7,
                edgecolor='k', linewidth=0.5, marker='o')
plt.title("km_driven vs Selling Price by Fuel Type", fontsize=16)
plt.xlabel("km_driven (km)", fontsize=12)
plt.ylabel("Selling Price", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(title='Fuel Type', fontsize=10, title_fontsize=12)
plt.show()
```



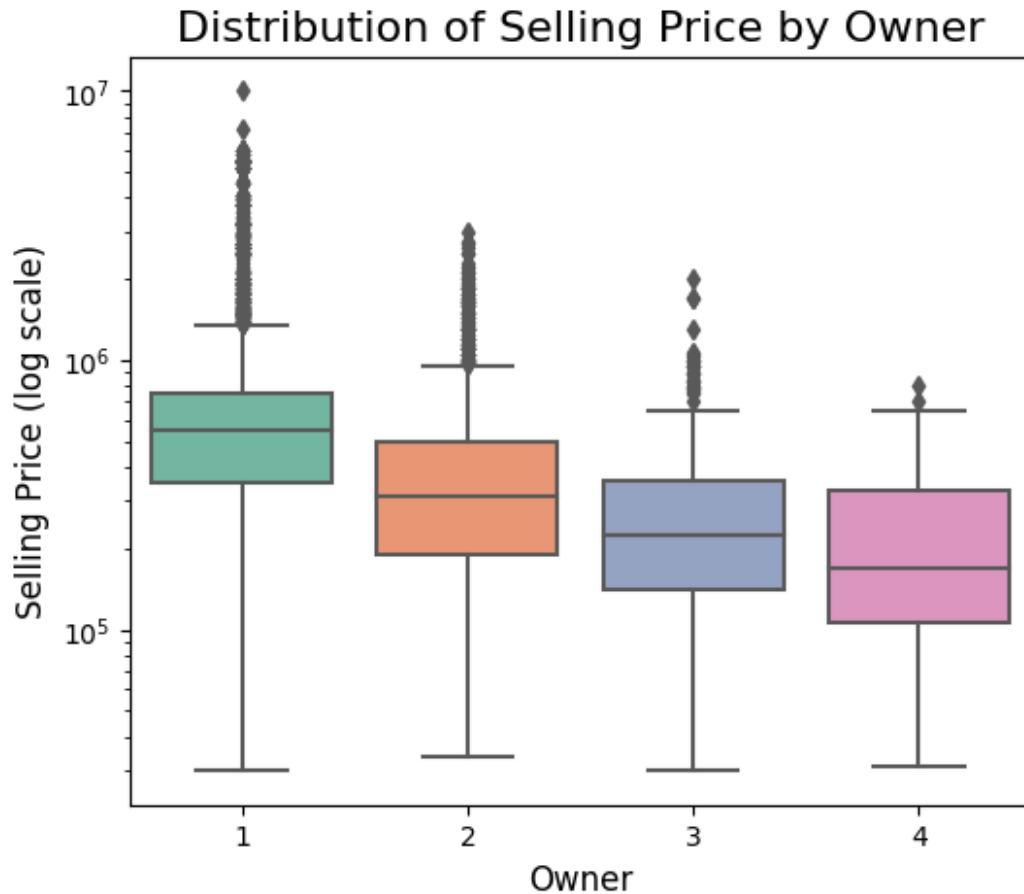
```
sns.scatterplot(data=df, x='max_power', y='selling_price', hue='fuel',
                sizes=(10, 20), palette='Set2', alpha=0.7,
                edgecolor='k', linewidth=0.5, marker='o')
plt.title("max_power vs Selling Price by Fuel Type", fontsize=16)
plt.xlabel("max_power", fontsize=12)
plt.ylabel("Selling Price", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(title='Fuel Type', fontsize=10, title_fontsize=12)
plt.show()
```



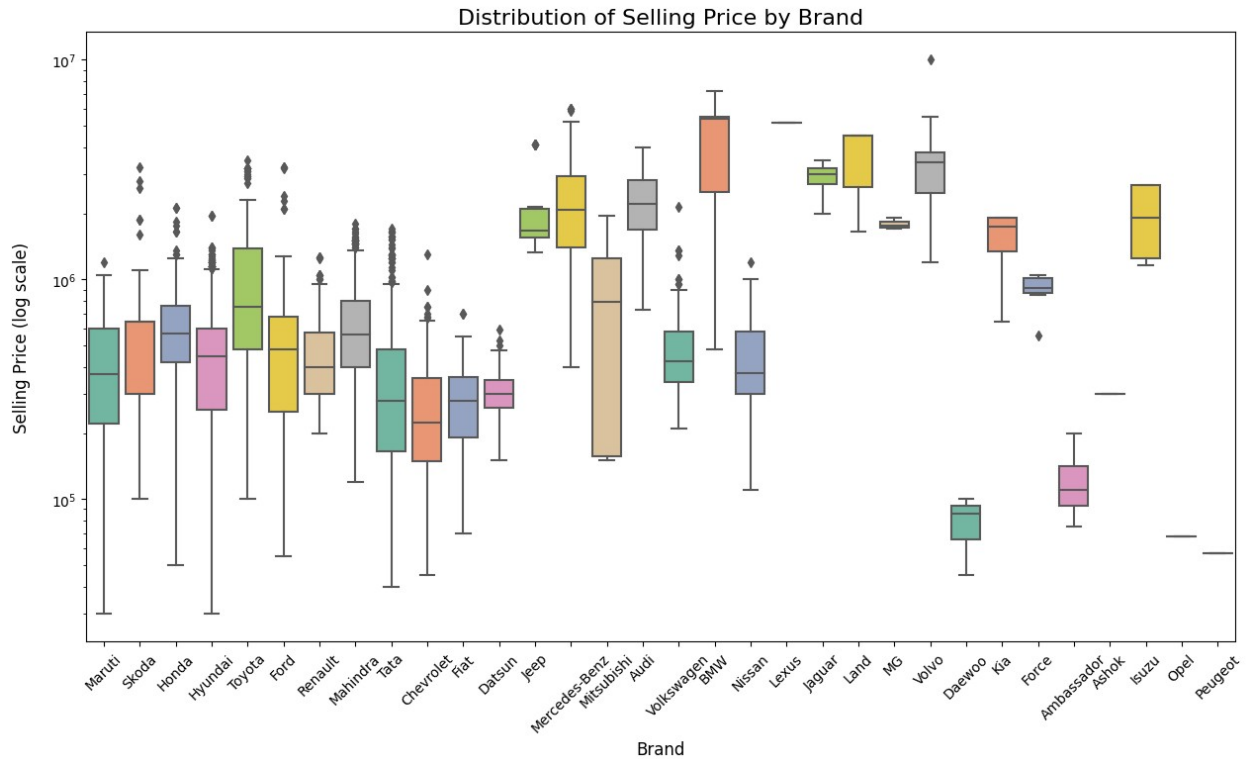
```
sns.scatterplot(data=df, x='seats', y='selling_price', hue='fuel',
                sizes=(10, 20), palette='Set2', alpha=0.7,
                edgecolor='k', linewidth=0.5, marker='o')
plt.title("seats vs Selling Price by Fuel Type", fontsize=16)
plt.xlabel("seats", fontsize=12)
plt.ylabel("Selling Price", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(title='Fuel Type', fontsize=10, title_fontsize=12)
plt.show()
```



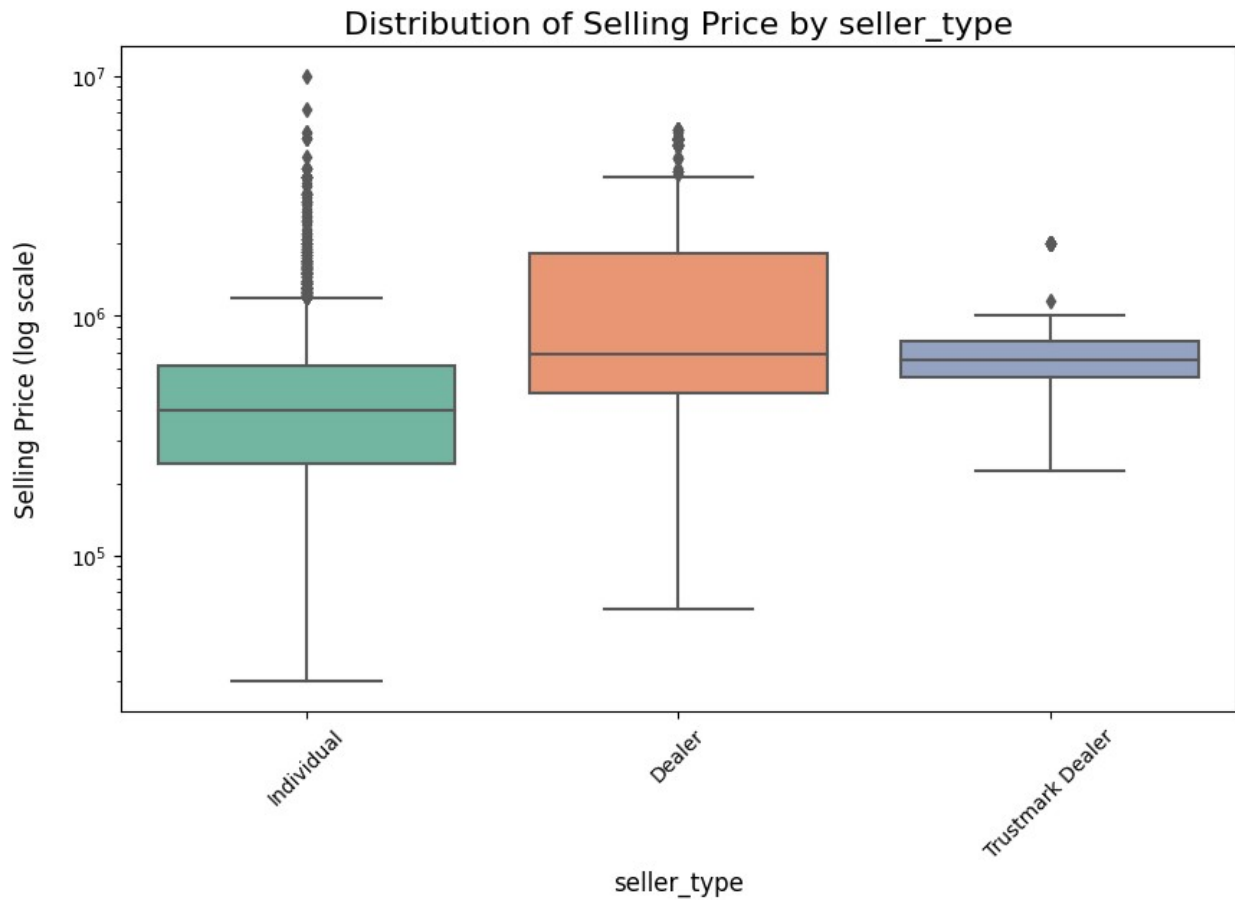
```
plt.figure(figsize=(6,5))
sns.boxplot(data=df, x='owner', y='selling_price', palette='Set2')
plt.yscale('log') # log scale to handle skewed values
plt.title("Distribution of Selling Price by Owner", fontsize=16)
plt.xlabel("Owner", fontsize=12)
plt.ylabel("Selling Price (log scale)", fontsize=12)
plt.show()
```



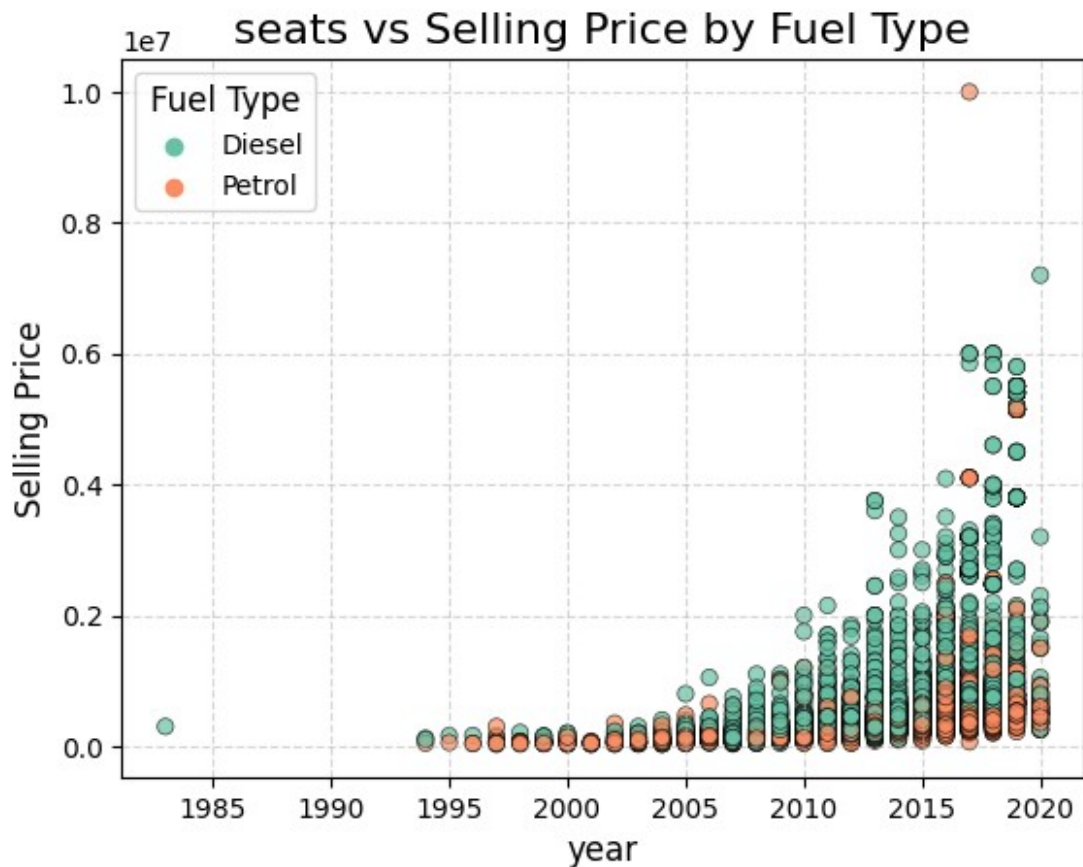
```
plt.figure(figsize=(15,8)) # make wider for readability
sns.boxplot(data=df, x='brand', y='selling_price', palette='Set2')
plt.yscale('log') # log scale to handle skewed values
plt.title("Distribution of Selling Price by Brand", fontsize=16)
plt.xlabel("Brand", fontsize=12)
plt.ylabel("Selling Price (log scale)", fontsize=12, rotation=90,
labelpad=15) # vertical label with padding
plt.xticks(rotation=45) # rotate x-axis labels if too crowded
plt.show()
```



```
plt.figure(figsize=(10,6)) # make wider for readability
sns.boxplot(data=df, x='seller_type', y='selling_price',
palette='Set2')
plt.yscale('log') # log scale to handle skewed values
plt.title("Distribution of Selling Price by seller_type", fontsize=16)
plt.xlabel("seller_type", fontsize=12)
plt.ylabel("Selling Price (log scale)", fontsize=12, rotation=90,
labelpad=15) # vertical label with padding
plt.xticks(rotation=45) # rotate x-axis labels if too crowded
plt.show()
```



```
sns.scatterplot(data=df, x='year', y='selling_price', hue='fuel',
                sizes=(10, 20), palette='Set2', alpha=0.7,
                edgecolor='k', linewidth=0.5, marker='o')
plt.title("seats vs Selling Price by Fuel Type", fontsize=16)
plt.xlabel("year", fontsize=12)
plt.ylabel("Selling Price", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(title='Fuel Type', fontsize=10, title_fontsize=12)
plt.show()
```



```
# Label Encoding Because I need to show the correlation
from sklearn.preprocessing import LabelEncoder
# brand_encoder = LabelEncoder()
# df['brand'] = brand_encoder.fit_transform(df['brand'])

fuel_encoder = LabelEncoder()
df['fuel'] = fuel_encoder.fit_transform(df['fuel'])

seller_type_encoder = LabelEncoder()
df['seller_type'] =
seller_type_encoder.fit_transform(df['seller_type'])

transmission_encoder = LabelEncoder()
df['transmission'] =
transmission_encoder.fit_transform(df['transmission'])

from sklearn.preprocessing import OneHotEncoder
import pandas as pd

def one_hot_transform(encoder, dataframe, feature):
    # Transform (already dense because sparse_output=False)
    encoded = encoder.transform(dataframe[[feature]])
```



```

# Get category names (skip first if drop='first')
categories = encoder.categories_[0]
if encoder.drop == 'first':
    categories = categories[1:]
encoded_df = pd.DataFrame(encoded, columns=[f"{feature}_{cat}" for
cat in categories], index=dataframe.index)

# Drop original column and concatenate
dataframe = dataframe.drop(columns=[feature])
dataframe = pd.concat([dataframe, encoded_df], axis=1)
return dataframe

```

Usage

```

brand_ohe = OneHotEncoder(drop='first',
sparse_output=False).fit(df[['brand']])
df = one_hot_transform(brand_ohe, df, 'brand')

```

df

	year	selling_price	km_driven	fuel	seller_type	transmission
owner \						
0	2014	450000	145500	0	1	1
1						
1	2014	370000	120000	0	1	1
2						
2	2006	158000	140000	1	1	1
3						
3	2010	225000	127000	0	1	1
1						
4	2007	130000	120000	1	1	1
1						
...	...	...	...	...	...	...
...						
8123	2013	320000	110000	1	1	1
1						
8124	2007	135000	119000	0	1	1
4						
8125	2009	382000	120000	0	1	1
1						
8126	2013	290000	25000	0	1	1
1						
8127	2013	290000	25000	0	1	1
1						
	mileage	engine	max_power	...	brand_Mitsubishi	brand_Nissan
\						
0	23.40	1248.0	74.00	...	0.0	0.0
1	21.14	1498.0	103.52	...	0.0	0.0

2	17.70	1497.0	78.00	...	0.0	0.0
3	23.00	1396.0	90.00	...	0.0	0.0
4	16.10	1298.0	88.20	...	0.0	0.0
...	...	...	...	...	...	...
8123	18.50	1197.0	82.85	...	0.0	0.0
8124	16.80	1493.0	110.00	...	0.0	0.0
8125	19.30	1248.0	73.90	...	0.0	0.0
8126	23.57	1396.0	70.00	...	0.0	0.0
8127	23.57	1396.0	70.00	...	0.0	0.0
brand_Opel brand_Peugeot brand_Renault brand_Skoda						
brand_Tata \						
0	0.0	0.0	0.0	0.0	0.0	
0.0						
1	0.0	0.0	0.0	0.0	1.0	
0.0						
2	0.0	0.0	0.0	0.0	0.0	
0.0						
3	0.0	0.0	0.0	0.0	0.0	
0.0						
4	0.0	0.0	0.0	0.0	0.0	
0.0						
...	...	...	...	...	...	..
.						
8123	0.0	0.0	0.0	0.0	0.0	
0.0						
8124	0.0	0.0	0.0	0.0	0.0	
0.0						
8125	0.0	0.0	0.0	0.0	0.0	
0.0						
8126	0.0	0.0	0.0	0.0	0.0	
1.0						
8127	0.0	0.0	0.0	0.0	0.0	
1.0						
brand_Toyota brand_Volkswagen brand_Volvo						
0	0.0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	0.0	

4	0.0	0.0	0.0
...	...	...	...
8123	0.0	0.0	0.0
8124	0.0	0.0	0.0
8125	0.0	0.0	0.0
8126	0.0	0.0	0.0
8127	0.0	0.0	0.0

[8028 rows x 42 columns]

```
df = df.drop(['index'], axis=1, errors='ignore')
```

```
df.head()
```

	year	selling_price	km_driven	fuel	seller_type	transmission
owner \						
0	2014	450000	145500	0	1	1
1						
1	2014	370000	120000	0	1	1
2						
2	2006	158000	140000	1	1	1
3						
3	2010	225000	127000	0	1	1
1						
4	2007	130000	120000	1	1	1
1						

	mileage	engine	max_power	...	brand_Mitsubishi	brand_Nissan	\
0	23.40	1248.0	74.00	...	0.0	0.0	
1	21.14	1498.0	103.52	...	0.0	0.0	
2	17.70	1497.0	78.00	...	0.0	0.0	
3	23.00	1396.0	90.00	...	0.0	0.0	
4	16.10	1298.0	88.20	...	0.0	0.0	

	brand_Opel	brand_Peugeot	brand_Renault	brand_Skoda	
brand_Tata \					
0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	1.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0

	brand_Toyota	brand_Volkswagen	brand_Volvo
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0

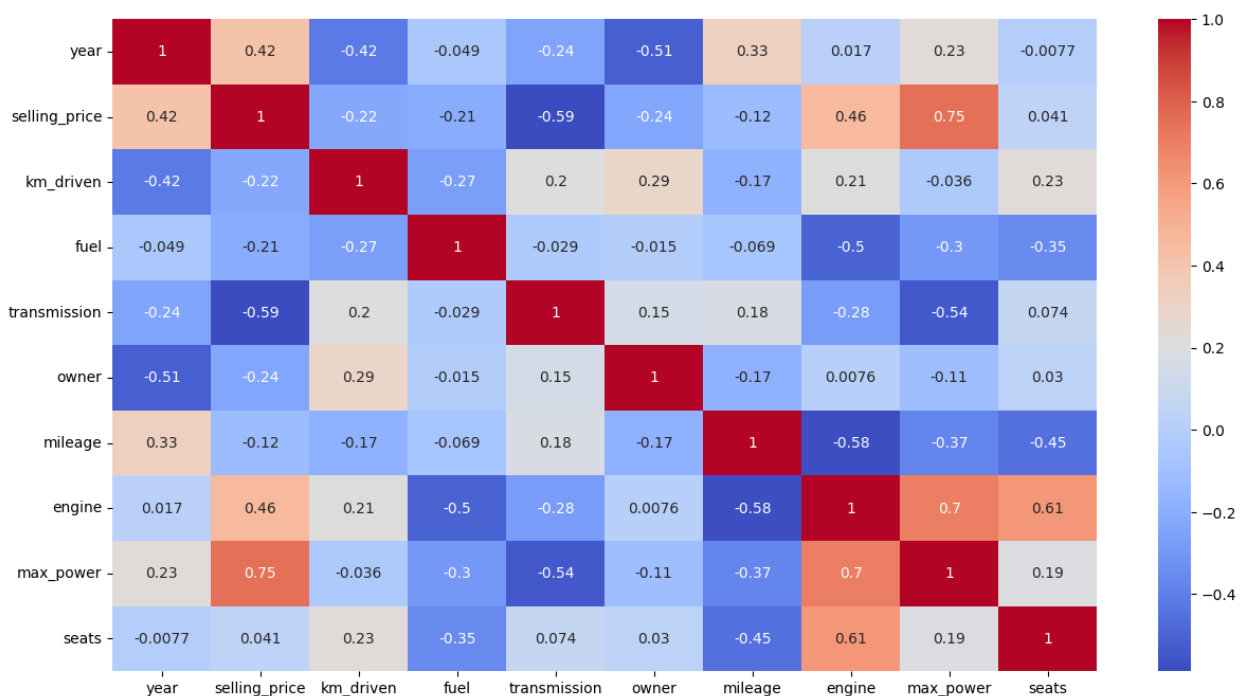
```
3          0.0          0.0          0.0
4          0.0          0.0          0.0
```

```
[5 rows x 42 columns]
```

```
# Excluding one-hot encoded columns
selected_cols = ['year', 'selling_price', 'km_driven', 'fuel',
                 'transmission', 'owner', 'mileage', 'engine', 'max_power', 'seats']
dfcopy = df.copy()
dfcopy = df[selected_cols]
```

```
plt.figure(figsize = (15,8))
sns.heatmap(dfcopy.corr(numeric_only=True), annot=True,
            cmap="coolwarm")
```

```
<Axes: >
```

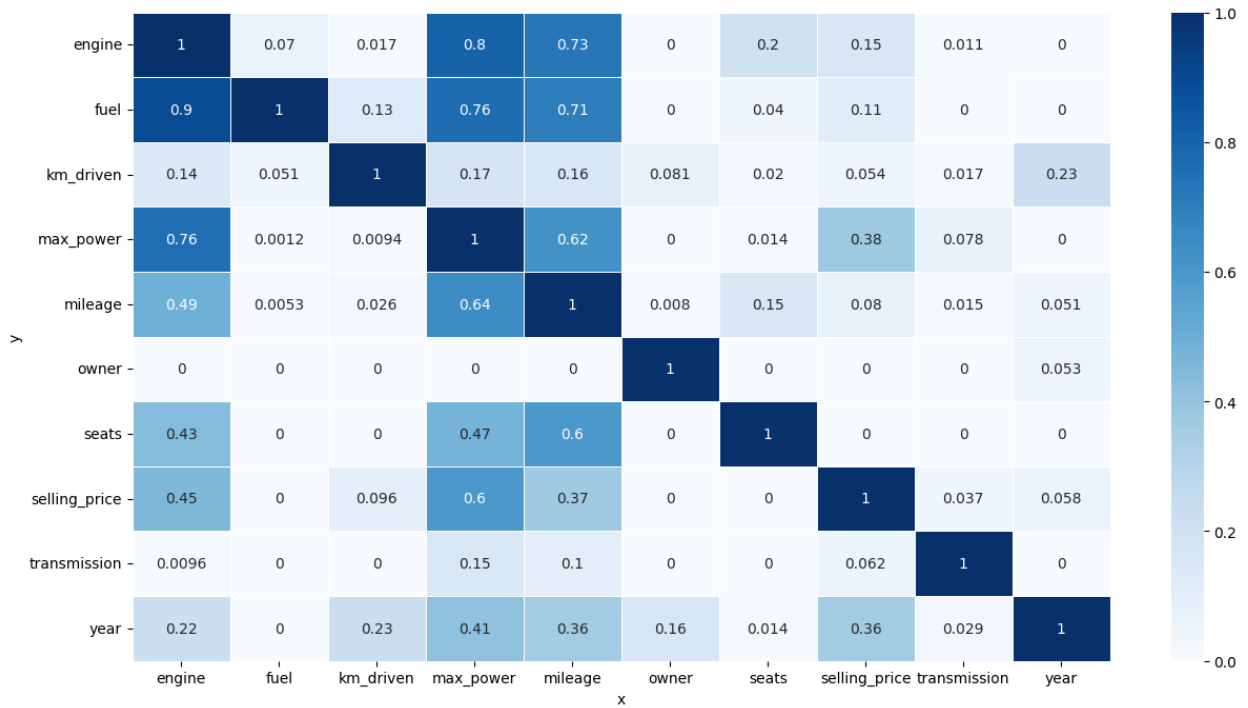


```
import ppscore as pps
```

```
matrix_df = pps.matrix(dfcopy)[['x', 'y',
                                  'ppscore']].pivot(columns='x', index='y', values='ppscore')
```

```
plt.figure(figsize = (15,8))
sns.heatmap(matrix_df, vmin=0, vmax=1, cmap="Blues", linewidths=0.5,
            annot=True)
```

```
<Axes: xlabel='x', ylabel='y'>
```



```
df['selling_price'] = pd.qcut(x=df['selling_price'], q=4,
labels=range(0, 4))
df.head()
```

	year	selling_price	km_driven	fuel	seller_type	transmission
owner \						
0	2014	1	145500	0	1	1
1						
1	2014	1	120000	0	1	1
2						
2	2006	0	140000	1	1	1
3						
3	2010	0	127000	0	1	1
1						
4	2007	0	120000	1	1	1
1						

	mileage	engine	max_power	...	brand_Mitsubishi	brand_Nissan	\
0	23.40	1248.0	74.00	...	0.0	0.0	
1	21.14	1498.0	103.52	...	0.0	0.0	
2	17.70	1497.0	78.00	...	0.0	0.0	
3	23.00	1396.0	90.00	...	0.0	0.0	
4	16.10	1298.0	88.20	...	0.0	0.0	

	brand_Opel	brand_Peugeot	brand_Renault	brand_Skoda	
brand_Tata \					
0	0.0	0.0	0.0	0.0	0.0

1	0.0	0.0	0.0	1.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0

	brand_Toyota	brand_Volkswagen	brand_Volvo
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0

[5 rows x 42 columns]

```
df['selling_price'].unique()
```

[1, 0, 2, 3]

Categories (4, int64): [0 < 1 < 2 < 3]

Get the column names after one-hot encoding

```
brand_cats = [f"brand_{cat}" for cat in brand_ohe.categories_[0][1:]]
```

Select features

```
selected_cols = ['max_power', 'year', 'fuel', 'mileage'] + brand_cats
```

```
X = df[selected_cols]
```

```
y = df['selling_price']
```

```
k = len(set(y))
```

```
k
```

4

```
df.head()
```

	year	selling_price	km_driven	fuel	seller_type	transmission
owner \						
0	2014	1	145500	0	1	1
1						
1	2014	1	120000	0	1	1
2						
2	2006	0	140000	1	1	1
3						
3	2010	0	127000	0	1	1
1						
4	2007	0	120000	1	1	1
1						
	mileage	engine	max_power	...	brand_Mitsubishi	brand_Nissan \

0	23.40	1248.0	74.00	...	0.0	0.0
1	21.14	1498.0	103.52	...	0.0	0.0
2	17.70	1497.0	78.00	...	0.0	0.0
3	23.00	1396.0	90.00	...	0.0	0.0
4	16.10	1298.0	88.20	...	0.0	0.0

	brand_Opel	brand_Peugeot	brand_Renault	brand_Skoda	
brand_Tata	\				
0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	1.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0

	brand_Toyota	brand_Volkswagen	brand_Volvo
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0

[5 rows x 42 columns]

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

null_train = X_train.isna().sum()
null_train[null_train > 0]

max_power    165
mileage      171
dtype: int64

null_test = X_test.isna().sum()
null_test[null_test > 0]

max_power    43
mileage      43
dtype: int64

y_train.isna().sum()

0

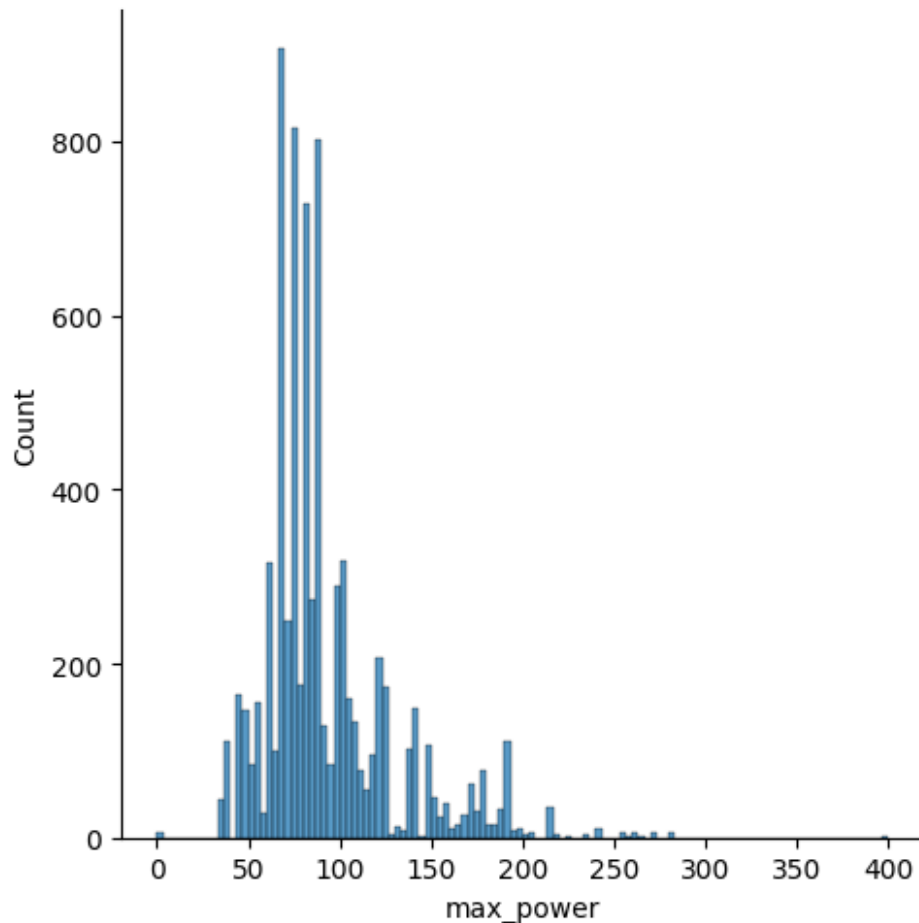
```

```
# Let's inspect the distribution plot, mean and median of max_power again
sns.displot(data=df, x='max_power')
```

```
print(f"Mean of max_power: {df['max_power'].mean()}")
print(f"Median of max_power: {df['max_power'].median()}")
```

Mean of max_power: 91.8197257033248

Median of max_power: 82.4

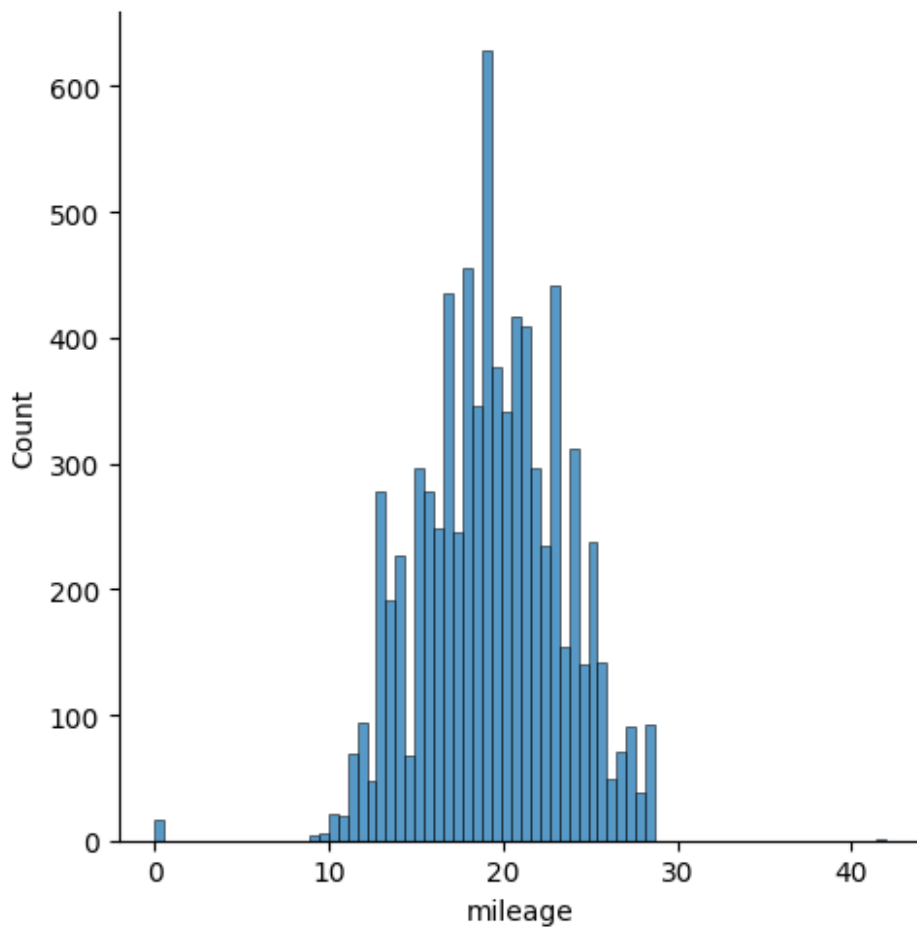


```
X_train['max_power'].fillna(X_train['max_power'].median(),
                             inplace=True)
X_test['max_power'].fillna(X_train['max_power'].median(),
                             inplace=True)
```

```
# Let's inspect the distribution plot, mean and median of mileage
sns.displot(data=df, x='mileage')
```

```
print(f"Mean of mileage: {df['mileage'].mean()}")
print(f"Median of mileage: {df['mileage'].median()}")
```


Mean of mileage: 19.391961863322244
Median of mileage: 19.3



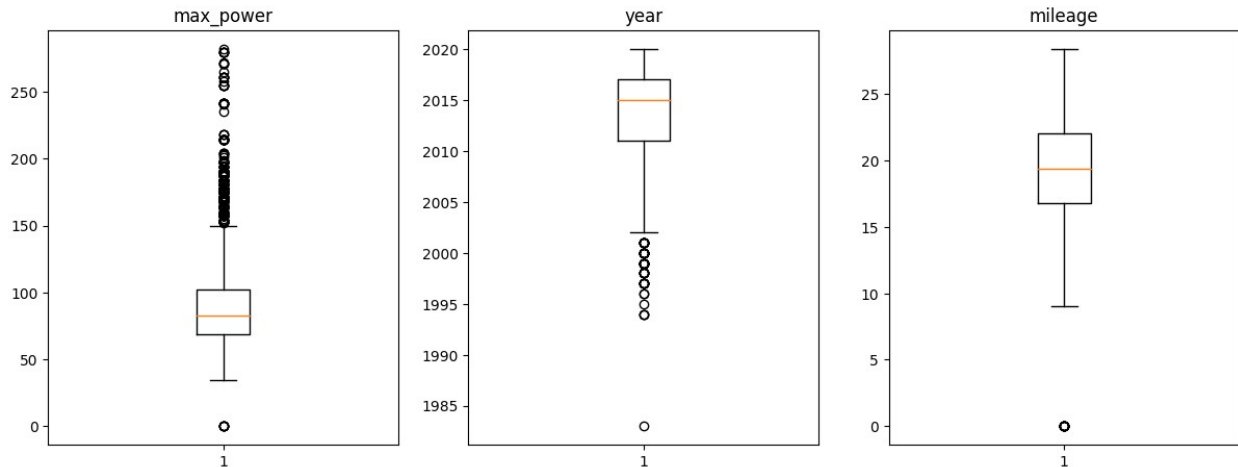
```
X_train['mileage'].fillna(X_train['mileage'].mean(), inplace=True)
X_test['mileage'].fillna(X_train['mileage'].mean(), inplace=True)

# Create a dictionary of columns.
col_dict = {'max_power': 1, 'year': 2, 'mileage': 3}

# Detect outliers in each variable using box plots.
plt.figure(figsize=(20,30))

for variable,i in col_dict.items():
    plt.subplot(5,4,i)
    plt.boxplot(X_train[variable])
    plt.title(variable)

plt.show()
```



```
def outlier_count(col, data = X_train):

    # calculate your 25% quatile and 75% quatile
    q75, q25 = np.percentile(data[col], [75, 25])

    # calculate your inter quatile
    iqr = q75 - q25

    # min_val and max_val
    min_val = q25 - (iqr*1.5)
    max_val = q75 + (iqr*1.5)

    # count number of outliers, which are the data that are less than
    # min_val or more than max_val calculated above
    outlier_count = len(np.where((data[col] > max_val) | (data[col] <
    min_val))[0])

    # calculate the percentage of the outliers
    outlier_percent = round(outlier_count/len(data[col])*100, 2)

    if(outlier_count > 0):
        print("\n"+15*'-'+ col + 15*'-'+ "\n")
        print('Number of outliers: {}'.format(outlier_count))
        print('Percent of data that is outlier: {}
    %'.format(outlier_percent))

    for col in X_train[['max_power', 'year', 'mileage']]:
        outlier_count(col)
```

-----max_power-----

Number of outliers: 459
Percent of data that is outlier: 7.15%

```

-----year-----
Number of outliers: 63
Percent of data that is outlier: 0.98%

-----mileage-----
Number of outliers: 16
Percent of data that is outlier: 0.25%

from sklearn.preprocessing import StandardScaler

# Define a list of numerical feature names that need to be scaled
num_cols = ['max_power', 'year', 'mileage']
scaler = StandardScaler()

X_train[num_cols] = scaler.fit_transform(X_train[num_cols])

X_test[num_cols] = scaler.transform(X_test[num_cols])

#Encoding of Y_train
Y_train_encoded = pd.get_dummies(y_train)

X_train = X_train.to_numpy()
X_test = X_test.to_numpy()
Y_train_encoded = Y_train_encoded.to_numpy()
y_test = y_test.to_numpy()

# Let's check shapes of all X_train, X_test, Y_train_encoded, y_test
print("Shape of X_train: ", X_train.shape)
print("Shape of X_test: ", X_test.shape)
print("Shape of y_train: ", Y_train_encoded.shape)
print("Shape of y_test: ", y_test.shape)

Shape of X_train: (6422, 35)
Shape of X_test: (1606, 35)
Shape of y_train: (6422, 4)
Shape of y_test: (1606,)

import time
from sklearn.metrics import classification_report
import mlflow

# mlflow.set_tracking_uri("https://mlflow.cs.ait.ac.th/")
# mlflow.set_experiment(experiment_name="st125985-a3")
import mlflow
import os

mlflow.set_tracking_uri(os.environ["MLFLOW_TRACKING_URI"])
mlflow.set_experiment("st125985-a3")

```

2025/09/19 02:35:27 INFO mlflow.tracking.fluent: Experiment with name 'st125985-a3' does not exist. Creating a new experiment.

```
<Experiment: artifact_location='mlflow-artifacts:/696234912322694663',
creation_time=1758249284173, experiment_id='696234912322694663',
last_update_time=1758249284173, lifecycle_stage='active',
name='st125985-a3', tags={}>
```

```
class LogisticRegression:
```

```
    def __init__(self, regularization, k, n, method, alpha=0.001,
max_iter=5000):
```

```
        self.regularization = regularization
        self.k = int(k)
        self.n = int(n)
        self.alpha = alpha
        self.max_iter = max_iter
        self.method = method
        self.W = None # Will be initialized later
        self.losses = []
```

```
    def fit(self, X, Y):
```

```
        """
        Fit the logistic regression model to the training data.
```

```
        Args:
```

```
            X (numpy.ndarray): Training data features.
            Y (numpy.ndarray): Training data labels.
```

```
        Returns:
```

```
            None
```

```
        """
```

```
        self.W = np.random.rand(self.n, self.k)
```

```
        params = {
            "reg": type(self).__name__,
            "method": self.method,
            "k": int(self.k),
            "n": int(self.n),
            "alpha": self.alpha,
            "max_iter": self.max_iter
        }
```

```
        mlflow.log_params(params=params)
```

```
        if self.method == "batch":
```

```
            start_time = time.time()
```

```
            for i in range(self.max_iter):
```

```
                loss, grad = self.gradient(X, Y)
```

```
                self.losses.append(loss)
```

```
                self.W = self.W - self.alpha * grad
```

```

        if i % 500 == 0:
            print(f"Loss at iteration {i}", loss)
            mlflow.log_metric(key="train_loss", value=loss,
step=i)
            print(f"time taken: {time.time() - start_time}")

        elif self.method == "minibatch":
            start_time = time.time()
            batch_size = int(0.3 * X.shape[0])
            for i in range(self.max_iter):
                ix = np.random.randint(0, X.shape[0]) # With
replacement
                batch_X = X[ix:ix + batch_size]
                batch_Y = Y[ix:ix + batch_size]
                loss, grad = self.gradient(batch_X, batch_Y)
                self.losses.append(loss)
                self.W = self.W - self.alpha * grad
                if i % 500 == 0:
                    print(f"Loss at iteration {i}", loss)
                    mlflow.log_metric(key="train_loss", value=loss,
step=i)
                    print(f"time taken: {time.time() - start_time}")

            elif self.method == "sto":
                start_time = time.time()
                list_of_used_ix = []
                for i in range(self.max_iter):
                    idx = np.random.randint(X.shape[0])
                    while i in list_of_used_ix:
                        idx = np.random.randint(X.shape[0])
                    X_train = X[idx, :].reshape(1, -1)
                    Y_train = Y[idx]
                    loss, grad = self.gradient(X_train, Y_train)
                    self.losses.append(loss)
                    self.W = self.W - self.alpha * grad

                    list_of_used_ix.append(i)
                    if len(list_of_used_ix) == X.shape[0]:
                        list_of_used_ix = []
                    if i % 500 == 0:
                        print(f"Loss at iteration {i}", loss)
                        mlflow.log_metric(key="train_loss", value=loss,
step=i)
                        print(f"time taken: {time.time() - start_time}")

                else:
                    raise ValueError('Method must be one of the followings:
"batch", "minibatch" or "sto".')

            def gradient(self, X, Y):

```

```

    """
    Compute the gradient and loss for the logistic regression
model.

    Args:
        X (numpy.ndarray): Input data features.
        Y (numpy.ndarray): Input data labels.

    Returns:
        float: Loss.
        numpy.ndarray: Gradient.
    """
    m = X.shape[0]
    h = self.h_theta(X, self.W)
    loss = -np.sum(Y * np.log(h)) / m
    error = h - Y

    if self.regularization:
        grad = self.softmax_grad(X, error) +
self.regularization.derivation(self.W)
    else:
        grad = self.softmax_grad(X, error)

    return loss, grad

def softmax(self, theta_t_x):
    """
    Compute the softmax probabilities.

    Args:
        theta_t_x (numpy.ndarray): Input data.

    Returns:
        numpy.ndarray: Softmax probabilities.
    """
    return np.exp(theta_t_x) / np.sum(np.exp(theta_t_x), axis=1,
keepdims=True)

def softmax_grad(self, X, error):
    """
    Compute the gradient for softmax regression.

    Args:
        X (numpy.ndarray): Input data features.
        error (numpy.ndarray): Error.

    Returns:
        numpy.ndarray: Gradient.
    """
    return X.T @ error

```

```

def h_theta(self, X, W):
    """
    Compute the predicted probabilities.

    Args:
        X (numpy.ndarray): Input data features.
        W (numpy.ndarray): Model weights.

    Returns:
        numpy.ndarray: Predicted probabilities.
    """
    return self.softmax(X @ W)

def predict(self, X_test):
    """
    Predict class labels for input data.

    Args:
        X_test (numpy.ndarray): Input data for prediction.

    Returns:
        numpy.ndarray: Predicted class labels.
    """
    return np.argmax(self.h_theta(X_test, self.W), axis=1)

def plot(self):
    """
    Plot the training losses.

    Returns:
        None
    """
    plt.plot(np.arange(len(self.losses)), self.losses,
label="Train Losses")
    plt.title("Losses")
    plt.xlabel("epoch")
    plt.ylabel("losses")
    plt.legend()

def accuracy(self, y_test, y_pred):
    """
    Compute classification accuracy.

    Args:
        y_test (numpy.ndarray): True class labels.
        y_pred (numpy.ndarray): Predicted class labels.

    Returns:
        float: Accuracy.
    """

```

```

    """
    correct_predictions = np.sum(y_test == y_pred)
    total_predictions = y_test.shape[0]
    return correct_predictions / total_predictions

def precision(self, y_test, y_pred, c=0):
    """
    Compute precision for a specific class `c`.

    Args:
        y_test (numpy.ndarray): True class labels.
        y_pred (numpy.ndarray): Predicted class labels.

    Returns:
        float: Precision.
    """
    true_positives = np.sum((y_test == c) & (y_pred == c))
    false_positives = np.sum((y_test != c) & (y_pred == c))
    if true_positives + false_positives == 0:
        return 0
    else:
        return true_positives / (true_positives + false_positives)

def recall(self, y_test, y_pred, c=0):
    """
    Compute recall for a specific class `c`.

    Args:
        y_test (numpy.ndarray): True class labels.
        y_pred (numpy.ndarray): Predicted class labels.

    Returns:
        float: Recall.
    """
    true_positives = np.sum((y_test == c) & (y_pred == c))
    false_negatives = np.sum((y_test == c) & (y_pred != c))
    if true_positives + false_negatives == 0:
        return 0
    else:
        return true_positives / (true_positives + false_negatives)

def f1_score(self, y_test, y_pred, c=0):
    """
    Compute F1-score for a specific class `c`.

    Args:
        y_test (numpy.ndarray): True class labels.
        y_pred (numpy.ndarray): Predicted class labels.

    Returns:

```



```

        float: F1-score.
    """
    precision = self.precision(y_test, y_pred, c)
    recall = self.recall(y_test, y_pred, c)
    if precision + recall == 0:
        return 0
    else:
        return 2 * precision * recall / (precision + recall)

def macro_precision(self, y_test, y_pred):
    """
    Compute macro-averaged precision.

    Args:
        y_test (numpy.ndarray): True class labels.
        y_pred (numpy.ndarray): Predicted class labels.

    Returns:
        float: Macro-averaged precision.
    """
    precisions = [self.precision(y_test, y_pred, c) for c in
range(self.k)]
    return np.sum(precisions) / self.k

def macro_recall(self, y_test, y_pred):
    """
    Compute macro-averaged recall.

    Args:
        y_test (numpy.ndarray): True class labels.
        y_pred (numpy.ndarray): Predicted class labels.

    Returns:
        float: Macro-averaged recall.
    """
    recalls = [self.recall(y_test, y_pred, c) for c in
range(self.k)]
    return np.sum(recalls) / self.k

def macro_f1(self, y_test, y_pred):
    """
    Compute macro-averaged F1-score.

    Args:
        y_test (numpy.ndarray): True class labels.
        y_pred (numpy.ndarray): Predicted class labels.

    Returns:
        float: Macro-averaged F1-score.
    """

```

```

        f1s = [self.f1_score(y_test, y_pred, c) for c in
range(self.k)]
        return np.sum(f1s) / self.k

    def weighted_precision(self, y_test, y_pred):
        """
        Compute weighted precision.

        Args:
            y_test (numpy.ndarray): True class labels.
            y_pred (numpy.ndarray): Predicted class labels.

        Returns:
            float: Weighted precision.
        """
        class_counts = [np.count_nonzero(y_test == c) for c in
range(self.k)]
        precisions = [class_counts[c] / len(y_test) *
self.precision(y_test, y_pred, c) for c in range(self.k)]
        return np.sum(precisions)

    def weighted_recall(self, y_test, y_pred):
        """
        Compute weighted recall.

        Args:
            y_test (numpy.ndarray): True class labels.
            y_pred (numpy.ndarray): Predicted class labels.

        Returns:
            float: Weighted recall.
        """
        class_counts = [np.count_nonzero(y_test == c) for c in
range(self.k)]
        recalls = [class_counts[c] / len(y_test) * self.recall(y_test,
y_pred, c) for c in range(self.k)]
        return np.sum(recalls)

    def weighted_f1(self, y_test, y_pred):
        """
        Compute weighted F1-score.

        Args:
            y_test (numpy.ndarray): True class labels.
            y_pred (numpy.ndarray): Predicted class labels.

        Returns:
            float: Weighted F1-score.
        """
        class_counts = [np.count_nonzero(y_test == c) for c in

```

```

range(self.k)]
    fls = [class_counts[c] / len(y_test) * self.f1_score(y_test,
y_pred, c) for c in range(self.k)]
    return np.sum(fls)

def classification_report(self, y_test, y_pred):
    """
    Generate a classification report.

    Args:
        y_test (numpy.ndarray): True class labels.
        y_pred (numpy.ndarray): Predicted class labels.

    Returns:
        pandas.DataFrame: Classification report.
    """
    cols = ["precision", "recall", "f1-score"]
    idx = list(range(self.k)) + ["accuracy", "macro", "weighted"]

    report = [[self.precision(y_test, y_pred, c),
                self.recall(y_test, y_pred, c),
                self.f1_score(y_test, y_pred, c)] for c in
range(self.k)]

    report.append(["", "", self.accuracy(y_test, y_pred)])

    report.append([self.macro_precision(y_test, y_pred),
                    self.macro_recall(y_test, y_pred),
                    self.macro_f1(y_test, y_pred)])

    report.append([self.weighted_precision(y_test, y_pred),
                    self.weighted_recall(y_test, y_pred),
                    self.weighted_f1(y_test, y_pred)])

    return pd.DataFrame(report, index=idx, columns=cols)

class RidgePenalty:
    """Ridge penalty (L2 regularization) for logistic regression.

    Args:
        l (float): Regularization strength.
    """

    def __init__(self, l):
        self.l = l

    def __call__(self, theta):
        """Compute the Ridge penalty term.

        Args:
            theta (numpy.ndarray): Model parameters.

```

```

    Returns:
        float: Ridge penalty term.
    """
    return self.l * np.sum(np.square(theta))

def derivation(self, theta):
    """Compute the derivative of the Ridge penalty.

    Args:
        theta (numpy.ndarray): Model parameters.

    Returns:
        numpy.ndarray: Derivative of the Ridge penalty.
    """
    return self.l * 2 * theta

class Ridge(LogisticRegression):
    """Logistic Regression with Ridge (L2) regularization.

    Args:
        l (float): Regularization strength.
        k (int): Number of classes.
        n (int): Number of features.
        method (str): Optimization method ('batch', 'minibatch', or
'sto').
        alpha (float, optional): Learning rate (default is 0.001).
        max_iter (int, optional): Maximum number of iterations
(default is 5000).
    """

    def __init__(self, l, k, n, method, alpha=0.001, max_iter=5000):
        regularization = RidgePenalty(l)
        super().__init__(regularization, k, n, method, alpha,
max_iter)

class Normal(LogisticRegression):
    """Logistic Regression without regularization.

    Args:
        k (int): Number of classes.
        n (int): Number of features.
        method (str): Optimization method ('batch', 'minibatch', or
'sto').
        alpha (float, optional): Learning rate (default is 0.001).
        max_iter (int, optional): Maximum number of iterations
(default is 5000).
    """

    def __init__(self, k, n, method, alpha=0.001, max_iter=5000):

```

```

        super().__init__(regularization=None, k=k, n=n, method=method,
alpha=alpha, max_iter=max_iter)

#helper function for looping classnames
import sys

def str_to_class(classname):
    return getattr(sys.modules[__name__], classname)

```

Performing experiment

```

# Grid search with regularization method (normal and), methods, and
alpha values

regs = ["Normal", "Ridge"]
methods = ["batch", "minibatch", "sto"]
alphas = [0.01, 0.001, 0.0001]
best_model = None
best_train_loss = float('inf')
best_reg_name = ""
best_method = ""

for reg in regs:
    for method in methods:
        mlflow.start_run(run_name=f"reg-{reg}-method-{method}",
nested=True)

        for alpha in alphas:
            if reg == "Normal":
                params = {"k": k, "n": X_train.shape[1], "method":
method, "alpha": alpha, "max_iter": 10000}
            else:
                params = {"k": k, "n": X_train.shape[1], "method":
method, "l": 0.1, "alpha": alpha, "max_iter": 10000}

            with mlflow.start_run(run_name=f"reg-{reg}-method-
{params['method']}-alpha-{params['alpha']}", nested=True):
                print("="*30)
                print(reg, method)
                print(f"alpha: {alpha}")
                print("="*30)
                type_of_regression = str_to_class(reg)      #Normal,
Ridge

                model = type_of_regression(**params)
                model.fit(X_train, Y_train_encoded)
                yhat = model.predict(X_test)

                accuracy = model.accuracy(y_test, yhat)

```

```

mlflow.log_metric(key="accuracy", value=accuracy)

for c in range(k):
    f1 = model.f1_score(y_test, yhat, c)
    precision = model.precision(y_test, yhat, c)
    recall = model.recall(y_test, yhat, c)
    mlflow.log_metric(key=f"class_{c}_f1", value=f1)
    mlflow.log_metric(key=f"class_{c}_recall",
value=recall)
    mlflow.log_metric(key=f"class_{c}_precision",
value=precision)

    # Get the final training loss
    final_train_loss = model.losses[-1]
    print(f"Final Training Loss: {final_train_loss}")

    # Compare the current model's training loss with the
best so far
    if final_train_loss < best_train_loss:
        best_train_loss = final_train_loss
        best_model = model
        best_reg_name = reg
        best_method = method

        signature = mlflow.models.infer_signature(X_train,
model.predict(X_train))
        mlflow.sklearn.log_model(model, artifact_path='model',
signature=signature)

        mlflow.end_run()

mlflow.end_run()

=====
Normal batch
alpha: 0.01
=====
Loss at iteration 0 1.2370153965742512
Loss at iteration 500 4.630724286912168
Loss at iteration 1000 4.595635872004949
Loss at iteration 1500 4.5889583150881155
Loss at iteration 2000 4.583315997876056
Loss at iteration 2500 4.569366374600114
Loss at iteration 3000 4.564482003418309
Loss at iteration 3500 4.56090432848851
Loss at iteration 4000 4.5589640352936085
Loss at iteration 4500 4.557607996739324
Loss at iteration 5000 4.554289185453395
Loss at iteration 5500 4.551932841237553
Loss at iteration 6000 4.549790358147496

```

```
Loss at iteration 6500 4.547304397120199
Loss at iteration 7000 4.546275730565748
Loss at iteration 7500 4.54583785977488
Loss at iteration 8000 4.545612172069022
Loss at iteration 8500 4.545476356000923
Loss at iteration 9000 4.545385207276153
Loss at iteration 9500 4.545319194445561
time taken: 28.009084939956665
Final Training Loss: 5.355486407291105
```

```
=====
Normal batch
alpha: 0.001
```

```
=====
Loss at iteration 0 1.3467857129292051
Loss at iteration 500 0.6029739968569889
Loss at iteration 1000 0.6013815882429316
Loss at iteration 1500 0.6006884058055713
Loss at iteration 2000 0.6002597733525112
Loss at iteration 2500 0.5999628876026712
Loss at iteration 3000 0.5997381930612807
Loss at iteration 3500 0.5995563370506913
Loss at iteration 4000 0.5994026849356348
Loss at iteration 4500 0.5992695144863333
Loss at iteration 5000 0.5991523801397981
Loss at iteration 5500 0.599048576703529
Loss at iteration 6000 0.5989565367927202
Loss at iteration 6500 0.5988755008915531
Loss at iteration 7000 0.5988051391308207
Loss at iteration 7500 0.5987451102416859
Loss at iteration 8000 0.5986947414201483
Loss at iteration 8500 0.5986529763092474
Loss at iteration 9000 0.5986185381976784
Loss at iteration 9500 0.5985901401574523
time taken: 29.626747369766235
Final Training Loss: 0.598566698463425
```

```
=====
Normal batch
alpha: 0.0001
```

```
=====
Loss at iteration 0 1.6277272339901316
Loss at iteration 500 0.6509830629445622
Loss at iteration 1000 0.6242203121866132
Loss at iteration 1500 0.6142346981789777
Loss at iteration 2000 0.6094681261453276
Loss at iteration 2500 0.6068811157635274
Loss at iteration 3000 0.6053452500437623
Loss at iteration 3500 0.6043642764764371
Loss at iteration 4000 0.6036965141282987
Loss at iteration 4500 0.6032156270735527
```

```
Loss at iteration 5000 0.6028518618586299
Loss at iteration 5500 0.6025649022876817
Loss at iteration 6000 0.6023304812399404
Loss at iteration 6500 0.6021334345103108
Loss at iteration 7000 0.6019639452969957
Loss at iteration 7500 0.601815441274209
Loss at iteration 8000 0.6016833798472557
Loss at iteration 8500 0.6015645260846875
Loss at iteration 9000 0.6014565116944232
Loss at iteration 9500 0.6013575584894312
time taken: 33.474973917007446
Final Training Loss: 0.6012664760860981
=====
```

```
Normal minibatch
alpha: 0.01
=====
```

```
Loss at iteration 0 1.3716114147669862
Loss at iteration 500 0.9233811593137206
Loss at iteration 1000 1.6958447135860375
Loss at iteration 1500 1.803853220321416
Loss at iteration 2000 1.3289011258150412
Loss at iteration 2500 0.9267480892102197
Loss at iteration 3000 0.8929219308342271
Loss at iteration 3500 1.093803093029221
Loss at iteration 4000 0.6489144133452248
Loss at iteration 4500 1.4288784027097727
Loss at iteration 5000 1.6554700219542229
Loss at iteration 5500 1.0020870289828638
Loss at iteration 6000 1.8082602642611478
Loss at iteration 6500 0.6793428429262325
Loss at iteration 7000 1.715537482886622
Loss at iteration 7500 1.065981274827036
Loss at iteration 8000 0.7204934920009722
Loss at iteration 8500 1.5494840715724367
Loss at iteration 9000 1.5722987827528523
Loss at iteration 9500 1.3751778463456825
time taken: 20.130431175231934
Final Training Loss: 1.5050543095058742
=====
```

```
Normal minibatch
alpha: 0.001
=====
```

```
Loss at iteration 0 1.3593530657805242
Loss at iteration 500 0.6173993036132571
Loss at iteration 1000 0.597461413154284
Loss at iteration 1500 0.5937527139117241
Loss at iteration 2000 0.5992132547653515
Loss at iteration 2500 0.6026164642634807
Loss at iteration 3000 0.5986385554068907
```



```
Loss at iteration 3500 0.5801689688959631
Loss at iteration 4000 0.6142129444760192
Loss at iteration 4500 0.6039668572236361
Loss at iteration 5000 0.6297193695132501
Loss at iteration 5500 0.6076101774589175
Loss at iteration 6000 0.6110196619688809
Loss at iteration 6500 0.5928717435441945
Loss at iteration 7000 0.5952068409617306
Loss at iteration 7500 0.5923636433378244
Loss at iteration 8000 0.5946007138912578
Loss at iteration 8500 0.5957713452742046
Loss at iteration 9000 0.613054005430652
Loss at iteration 9500 0.632186668840335
time taken: 17.011264085769653
Final Training Loss: 0.611941786471475
```

```
=====
Normal minibatch
alpha: 0.0001
```

```
=====
Loss at iteration 0 1.662117870711096
Loss at iteration 500 0.7691957242514851
Loss at iteration 1000 0.6962962545617888
Loss at iteration 1500 0.6786249685224552
Loss at iteration 2000 0.6477340586626127
Loss at iteration 2500 0.6286756637509178
Loss at iteration 3000 0.6271923098966449
Loss at iteration 3500 0.6215974572958259
Loss at iteration 4000 0.6134130089401375
Loss at iteration 4500 0.6278998120458928
Loss at iteration 5000 0.6091157727061953
Loss at iteration 5500 0.6099028237329722
Loss at iteration 6000 0.604757213419246
Loss at iteration 6500 0.6006799192901617
Loss at iteration 7000 0.6061287182305999
Loss at iteration 7500 0.5901350122332416
Loss at iteration 8000 0.6194873287103504
Loss at iteration 8500 0.6121568725179073
Loss at iteration 9000 0.6124626720048455
Loss at iteration 9500 0.5953961294453191
time taken: 18.585987091064453
Final Training Loss: 0.6091706939104209
```

```
=====
Normal sto
alpha: 0.01
```

```
=====
Loss at iteration 0 1.241884805401242
Loss at iteration 500 1.1539555234352923
Loss at iteration 1000 1.6343011902943374
Loss at iteration 1500 1.4313188401654757
```

```
Loss at iteration 2000 0.8625761172546016
Loss at iteration 2500 1.601113194340161
Loss at iteration 3000 0.11712748834028533
Loss at iteration 3500 0.19653529228492456
Loss at iteration 4000 1.2011567127301213
Loss at iteration 4500 0.06656321788402664
Loss at iteration 5000 0.46905622252365864
Loss at iteration 5500 0.8247630567236806
Loss at iteration 6000 0.17734972714489888
Loss at iteration 6500 0.8324633392941052
Loss at iteration 7000 0.2219972242471584
Loss at iteration 7500 1.2230155899365656
Loss at iteration 8000 0.2581082972008568
Loss at iteration 8500 0.3396564086885774
Loss at iteration 9000 0.828840517655856
Loss at iteration 9500 3.7288168824908943
time taken: 4.997492074966431
Final Training Loss: 0.2487343941960908
```

```
=====
Normal sto
alpha: 0.001
```

```
=====
Loss at iteration 0 0.5430410493480281
Loss at iteration 500 1.1178082576701864
Loss at iteration 1000 1.4784186338278429
Loss at iteration 1500 0.35962167050519406
Loss at iteration 2000 0.563488546538656
Loss at iteration 2500 1.6518959160408726
Loss at iteration 3000 1.460549468961869
Loss at iteration 3500 0.3942663142939303
Loss at iteration 4000 0.3711546988165023
Loss at iteration 4500 0.5882643828207084
Loss at iteration 5000 0.3651981219637716
Loss at iteration 5500 1.055559647276592
Loss at iteration 6000 0.9382383922317235
Loss at iteration 6500 1.392402333211018
Loss at iteration 7000 1.5956585566954522
Loss at iteration 7500 1.3182639905905222
Loss at iteration 8000 0.7970629310763525
Loss at iteration 8500 0.1830195090850513
Loss at iteration 9000 1.484948324167569
Loss at iteration 9500 1.6754259631375341
time taken: 4.654280185699463
Final Training Loss: 1.2423838250278607
```

```
=====
Normal sto
alpha: 0.0001
```

```
=====
Loss at iteration 0 1.6269401723354704
```

```
Loss at iteration 500 1.3494159964650898
Loss at iteration 1000 1.6876751279024003
Loss at iteration 1500 1.4873896203626453
Loss at iteration 2000 0.8377798096629244
Loss at iteration 2500 1.3311565850130662
Loss at iteration 3000 0.6170937804024262
Loss at iteration 3500 1.0342974534476348
Loss at iteration 4000 1.0162547596970588
Loss at iteration 4500 0.6535159169036038
Loss at iteration 5000 1.2027735411266465
Loss at iteration 5500 1.1741478137068009
Loss at iteration 6000 0.819204166807771
Loss at iteration 6500 1.4484776701403352
Loss at iteration 7000 0.8362689864892152
Loss at iteration 7500 1.7022560505549709
Loss at iteration 8000 1.1134026773108796
Loss at iteration 8500 1.7851312473323613
Loss at iteration 9000 0.9457186630434627
Loss at iteration 9500 1.1855297046016045
time taken: 4.76997447013855
Final Training Loss: 1.1389881435858638
```

```
=====
Ridge batch
alpha: 0.01
```

```
=====
Loss at iteration 0 1.5569345079957009
Loss at iteration 500 4.918175742466359
Loss at iteration 1000 4.901049696062366
Loss at iteration 1500 4.899943363004794
Loss at iteration 2000 4.899788413579942
Loss at iteration 2500 4.899748836615055
Loss at iteration 3000 4.899736281213172
Loss at iteration 3500 4.899731983119652
Loss at iteration 4000 4.899730459669113
Loss at iteration 4500 4.899729909534406
Loss at iteration 5000 4.899729708775532
Loss at iteration 5500 4.899729635078265
Loss at iteration 6000 4.899729607937308
Loss at iteration 6500 4.899729597925701
Loss at iteration 7000 4.8997295942301005
Loss at iteration 7500 4.8997295928657
Loss at iteration 8000 4.899729592362027
Loss at iteration 8500 4.89972959217614
Loss at iteration 9000 4.899729592107559
Loss at iteration 9500 4.899729592082266
time taken: 33.710684299468994
Final Training Loss: 5.508775630470081
```

```
=====
Ridge batch
```

alpha: 0.001

```
=====
Loss at iteration 0 1.4661027676921603
Loss at iteration 500 0.6036112860523134
Loss at iteration 1000 0.601927269029298
Loss at iteration 1500 0.601288440616335
Loss at iteration 2000 0.60092630861458
Loss at iteration 2500 0.6006932084401687
Loss at iteration 3000 0.6005319368239209
Loss at iteration 3500 0.6004146305089444
Loss at iteration 4000 0.6003261403097905
Loss at iteration 4500 0.600257572756918
Loss at iteration 5000 0.6002033697344428
Loss at iteration 5500 0.6001598682160699
Loss at iteration 6000 0.6001245434955942
Loss at iteration 6500 0.6000955898701192
Loss at iteration 7000 0.6000716767709265
Loss at iteration 7500 0.6000518002488914
Loss at iteration 8000 0.6000351884859344
Loss at iteration 8500 0.6000212392039177
Loss at iteration 9000 0.6000094766964852
Loss at iteration 9500 0.5999995214242827
time taken: 50.28269672393799
Final Training Loss: 0.5999910835381247
=====
```

Ridge batch

alpha: 0.0001

```
=====
Loss at iteration 0 1.6031973823812387
Loss at iteration 500 0.654109073544043
Loss at iteration 1000 0.6256207490274404
Loss at iteration 1500 0.6152425229651947
Loss at iteration 2000 0.610338394569156
Loss at iteration 2500 0.6076762482224997
Loss at iteration 3000 0.6060849393881889
Loss at iteration 3500 0.6050586455744238
Loss at iteration 4000 0.6043535186872396
Loss at iteration 4500 0.6038424264810442
Loss at iteration 5000 0.6034548946819182
Loss at iteration 5500 0.6031497921791674
Loss at iteration 6000 0.6029020014799271
Loss at iteration 6500 0.6026955449208229
Loss at iteration 7000 0.6025198744697022
Loss at iteration 7500 0.6023677881279955
Loss at iteration 8000 0.6022342171214351
Loss at iteration 8500 0.6021154960630833
Loss at iteration 9000 0.6020089097787397
Loss at iteration 9500 0.6019124034880797
time taken: 44.15212368965149
```

Final Training Loss: 0.6018245606871209

=====

Ridge minibatch

alpha: 0.01

=====

Loss at iteration 0 1.361109322713749
Loss at iteration 500 0.7779710037124935
Loss at iteration 1000 1.2854044996584475
Loss at iteration 1500 1.7225930113651786
Loss at iteration 2000 1.8657247353737199
Loss at iteration 2500 1.8755608607367114
Loss at iteration 3000 1.9770927560505254
Loss at iteration 3500 1.5281840748217166
Loss at iteration 4000 1.8185533161718235
Loss at iteration 4500 1.7515851579706754
Loss at iteration 5000 1.6464366242724424
Loss at iteration 5500 1.66523140294761
Loss at iteration 6000 1.4328190899686888
Loss at iteration 6500 0.6478506387481894
Loss at iteration 7000 0.9129338146596067
Loss at iteration 7500 1.309810869436869
Loss at iteration 8000 1.779480465945787
Loss at iteration 8500 1.6177901580784848
Loss at iteration 9000 0.8169172828517589
Loss at iteration 9500 2.1843512552054385
time taken: 17.56306505203247

Final Training Loss: 0.8675624549040815

=====

Ridge minibatch

alpha: 0.001

=====

Loss at iteration 0 1.605114950162632
Loss at iteration 500 0.6167296236609968
Loss at iteration 1000 0.6372407640692135
Loss at iteration 1500 0.6095313868585168
Loss at iteration 2000 0.6113229907570941
Loss at iteration 2500 0.5795207740628093
Loss at iteration 3000 0.6209893357250698
Loss at iteration 3500 0.608051351799177
Loss at iteration 4000 0.6602563241926819
Loss at iteration 4500 0.5778522817288028
Loss at iteration 5000 0.6207106758104834
Loss at iteration 5500 0.6207028592000237
Loss at iteration 6000 0.58454376700702
Loss at iteration 6500 0.6072629580429556
Loss at iteration 7000 0.6020635925435096
Loss at iteration 7500 0.6084098019256483
Loss at iteration 8000 0.5857641390027976
Loss at iteration 8500 0.6080525077647796

```
Loss at iteration 9000 0.5852598894730441
Loss at iteration 9500 0.6053501144821278
time taken: 35.55305051803589
Final Training Loss: 0.6140408631559748
```

```
=====
```

```
Ridge minibatch
```

```
alpha: 0.0001
```

```
=====
```

```
Loss at iteration 0 1.6631305693189566
Loss at iteration 500 0.7434015011399333
Loss at iteration 1000 0.6905929425382197
Loss at iteration 1500 0.6544942967419354
Loss at iteration 2000 0.665454211963775
Loss at iteration 2500 0.6557203495967917
Loss at iteration 3000 0.6427961576650348
Loss at iteration 3500 0.6217565434491876
Loss at iteration 4000 0.6468242863707321
Loss at iteration 4500 0.6322047565542964
Loss at iteration 5000 0.6114910225699675
Loss at iteration 5500 0.610393346359732
Loss at iteration 6000 0.6087704478309418
Loss at iteration 6500 0.6307184714102885
Loss at iteration 7000 0.6010261485898046
Loss at iteration 7500 0.6097598645722024
Loss at iteration 8000 0.6174565678813719
Loss at iteration 8500 0.6155384330697206
Loss at iteration 9000 0.6134027409867087
Loss at iteration 9500 0.6055806689867597
time taken: 13.952974796295166
Final Training Loss: 0.6536057589188808
```

```
=====
```

```
Ridge sto
```

```
alpha: 0.01
```

```
=====
```

```
Loss at iteration 0 2.1824610662917827
Loss at iteration 500 0.4423733646947648
Loss at iteration 1000 1.2577350087732497
Loss at iteration 1500 1.6983568532584434
Loss at iteration 2000 1.2532546786361132
Loss at iteration 2500 1.6140793454554105
Loss at iteration 3000 0.42502304130935864
Loss at iteration 3500 1.7901082012282434
Loss at iteration 4000 1.4459422365718648
Loss at iteration 4500 1.3780040600670669
Loss at iteration 5000 1.0844909943185383
Loss at iteration 5500 1.4682637715505846
Loss at iteration 6000 0.37707959695522447
Loss at iteration 6500 1.1373022654092195
Loss at iteration 7000 1.3093504889301206
```

```
Loss at iteration 7500 1.544312121266659
Loss at iteration 8000 0.8840005339302888
Loss at iteration 8500 1.290446748627066
Loss at iteration 9000 1.0792023966590372
Loss at iteration 9500 1.457784982403599
time taken: 6.0513365268707275
Final Training Loss: 0.5760614636311374
=====
```

```
Ridge sto
alpha: 0.001
=====
```

```
Loss at iteration 0 2.182242651207618
Loss at iteration 500 1.4799106015157373
Loss at iteration 1000 1.0303878314744974
Loss at iteration 1500 1.745838274696536
Loss at iteration 2000 1.3941049712269447
Loss at iteration 2500 0.6882632494983023
Loss at iteration 3000 0.8059138742071534
Loss at iteration 3500 0.9347328525890024
Loss at iteration 4000 1.2436728548475604
Loss at iteration 4500 0.5793199953828757
Loss at iteration 5000 1.319930540212016
Loss at iteration 5500 1.03718545377998
Loss at iteration 6000 1.5594539502016145
Loss at iteration 6500 1.3521229633124967
Loss at iteration 7000 1.1762175712088656
Loss at iteration 7500 0.7715702363089777
Loss at iteration 8000 1.2617186997175143
Loss at iteration 8500 1.0698817050505378
Loss at iteration 9000 1.4049891078051195
Loss at iteration 9500 1.0686977055755462
time taken: 4.506785869598389
Final Training Loss: 0.5984836681115286
=====
```

```
Ridge sto
alpha: 0.0001
=====
```

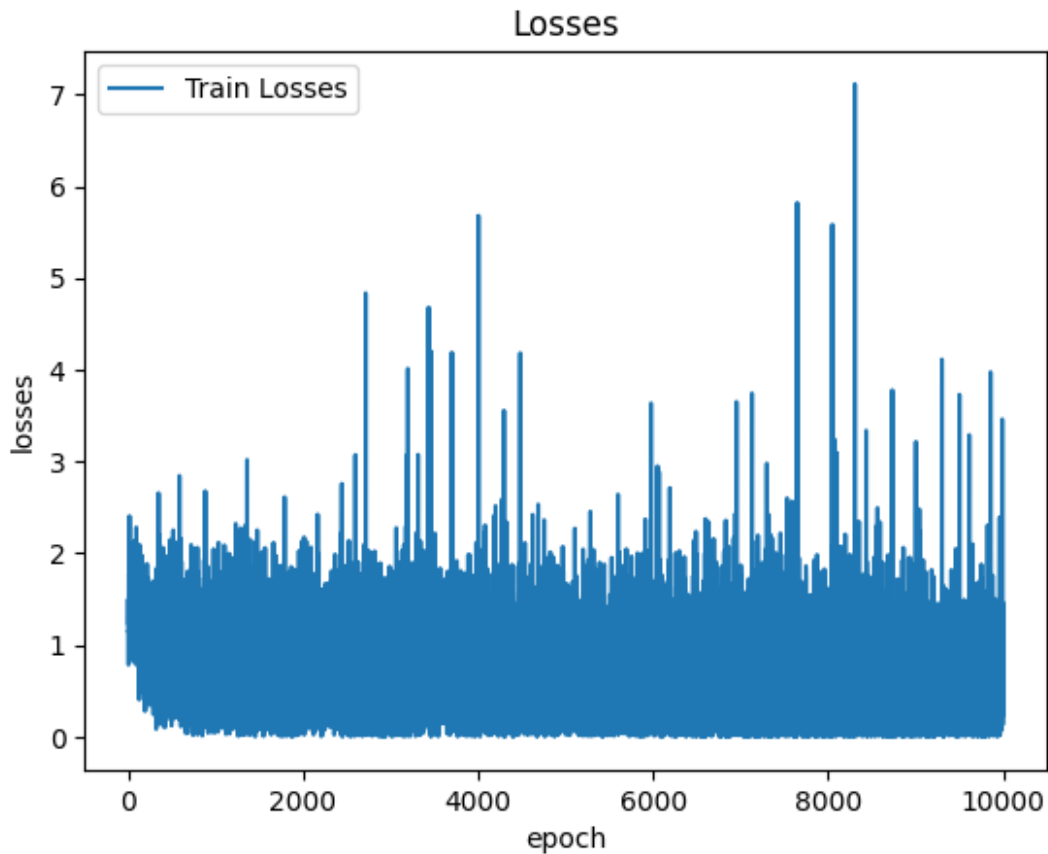
```
Loss at iteration 0 0.8193053702299592
Loss at iteration 500 1.5224087917607239
Loss at iteration 1000 1.5639288439550638
Loss at iteration 1500 2.1512230270078825
Loss at iteration 2000 1.7204143914108037
Loss at iteration 2500 2.126945027493673
Loss at iteration 3000 1.5103254842343052
Loss at iteration 3500 1.8784969488524832
Loss at iteration 4000 0.2985576037202347
Loss at iteration 4500 0.6556374772749132
Loss at iteration 5000 1.2598940312637659
Loss at iteration 5500 0.9489609499528642
```

```
Loss at iteration 6000 1.3011596761485513
Loss at iteration 6500 1.5519794237857105
Loss at iteration 7000 1.653256905224131
Loss at iteration 7500 1.4649412688185048
Loss at iteration 8000 1.6060783303593793
Loss at iteration 8500 0.5432569196741909
Loss at iteration 9000 0.8881322107426237
Loss at iteration 9500 0.8708598982171443
time taken: 4.527048110961914
Final Training Loss: 1.3020416253870046
```

Finding the best model and comparing its confusion matrix with sklearn

```
# Plotting the loss graph the graph of the best model
print("=" * 40)
print("Best Model:")
print(f"Regularization: {best_reg_name}")
print(f"Method: {best_method}")
print(f"Final Training Loss: {best_train_loss}")
print("=" * 40)
best_model.plot()
plt.show()
```

```
=====
Best Model:
Regularization: Normal
Method: sto
Final Training Loss: 0.2487343941960908
=====
```

```
from sklearn.metrics import classification_report as
sklearn_classification_report

# Assuming best_model is already defined based on the previous code

# Use the best model to predict
yhat = best_model.predict(X_test)

# Custom classification report
custom_classification_report =
best_model.classification_report(y_test, yhat)
print("'" * 5, "Custom Classification report", "'" * 5)
print(custom_classification_report)

# Sklearn's classification report
sklearn_report = sklearn_classification_report(y_test, yhat)
print("\n")
print("'" * 5, "Sklearn's Classification report", "'" * 5)
print(sklearn_report)

***** Custom Classification report *****
           precision    recall  f1-score
0               0.743852    0.883212    0.807564
```

1	0.656642	0.574561	0.612865
2	0.556911	0.389205	0.458194
3	0.716702	0.875969	0.788372
accuracy			0.685554
macro	0.668527	0.680737	0.666749
weighted	0.671574	0.685554	0.671083

***** Sklearn's Classification report *****

	precision	recall	f1-score	support
0	0.74	0.88	0.81	411
1	0.66	0.57	0.61	456
2	0.56	0.39	0.46	352
3	0.72	0.88	0.79	387
accuracy			0.69	1606
macro avg	0.67	0.68	0.67	1606
weighted avg	0.67	0.69	0.67	1606

The support in the Sklearn's Classification Report shows that the test dataset has four categories. The 0 category has 411 samples, 1 category has 456 samples, 2 category has 352 samples and category 3 has 387 samples.

Testing the model importing from MLFlow

```
import pickle

# paths of all components for car price predictions
brand_label_encoder_path = 'Model/car_brand_encoder.model'
scaler_path = 'Model/car-scaling.model'
fuel_label_encoder_path = 'Model/car_fuel_encoder.model'
pickle.dump(brand_ohe, open(brand_label_encoder_path, 'wb'))
pickle.dump(scaler, open(scaler_path, 'wb'))
pickle.dump(fuel_encoder, open(fuel_label_encoder_path, 'wb'))

print(model)

<__main__.Ridge object at 0x7f5e5f70e350>
```

Registration model with the run id and model name

```
# To view all the metrics
from mlflow.tracking import MlflowClient
```

```

# client = MlflowClient()
# runs = client.search_runs(experiment_ids=[469864923438442098])

# for r in runs:
#     print(f"Run ID: {r.info.run_id}")
#     print("Metrics:", r.data.metrics)

# ## Deleting the model
# from mlflow.tracking import MlflowClient

# client = MlflowClient()
# model_name = "st125985-a3-model"

# # Delete all versions first
# for mv in client.get_latest_versions(model_name):
#     client.delete_model_version(name=model_name, version=mv.version)

# # Delete the registered model itself
# client.delete_registered_model(name=model_name)

# print(f"Deleted model '{model_name}' successfully!")

## Registering the model with best run id
import mlflow
from mlflow.tracking import MlflowClient

# MLflow client
client = MlflowClient()

# Your new model name
model_name = "st125985-a3-model"

# Best run info
best_run_id = "6455edd4b0324ef48854fac452346c69"
model_uri = f"runs://{best_run_id}/model" # Assuming your model
artifact is saved under "model"

# Check if the model already exists and delete if needed
try:
    client.delete_registered_model(name=model_name)
    print(f"Deleted existing registered model '{model_name}'")
except Exception as e:
    print(f"No existing model to delete or error: {e}")

# Register the new model
registered_model = mlflow.register_model(model_uri=model_uri,
name=model_name)

```

```
print(f"Registered new model '{model_name}' with version:
{registered_model.version}")
```

No existing model to delete or error: RESOURCE_DOES_NOT_EXIST:
Registered Model with name=st125985-a3-model not found

Successfully registered model 'st125985-a3-model'.
2025/09/19 03:00:47 INFO mlflow.tracking._model_registry.client:
Waiting up to 300 seconds for model version to finish creation. Model
name: st125985-a3-model, version 1

Registered new model 'st125985-a3-model' with version: 1

Created version '1' of model 'st125985-a3-model'.

```
from mlflow.tracking import MlflowClient
```

```
client = MlflowClient()
```

```
for mv in client.get_latest_versions("st125985-a3-model"):
    print(f"Version: {mv.version}, Stage: {mv.current_stage}")
```

Version: 1, Stage: None

Model promoted to Production!

```
from mlflow.tracking import MlflowClient
```

```
client = MlflowClient()
```

Promote version 1 to Production

```
client.transition_model_version_stage(
    name="st125985-a3-model",
    version=1,
    stage="Production",
    archive_existing_versions=True # optional: archive any other
Production version
)
```

```
print("Model promoted to Production!")
```

Model promoted to Production!

Loading the model from model registry

```
import mlflow
```

```
run_id = "6455edd4b0324ef48854fac452346c69"
```

```
model = mlflow.pyfunc.load_model(f"runs://{run_id}/model")
```

```
{"model_id": "896b2324ca0348cfad7e26c37660dbce", "version_major": 2, "vers
ion_minor": 0}
```

```
# import mlflow
```

```
# run_id = "671aac8734d64f77b16755dee5182a4f"
```

```

# client = mlflow.tracking.MlflowClient()

# # List all artifacts in the run
# artifacts = client.list_artifacts(run_id)
# for a in artifacts:
#     print(a.path)

model

# # Loading directly from a Run
# best_run_id = "671aac8734d64f77b16755dee5182a4f"
# model_uri = f"runs:{best_run_id}/model"
# model = mlflow.pyfunc.load_model(model_uri)

# predict X_test based on the loaded model
import pandas as pd
precited_selling_price= model.predict(pd.DataFrame(X_test))
precited_selling_price[:10]

array([0, 3, 1, 2, 3, 0, 1, 1, 1, 1])

# The actual values of y
y_test[:10]

array([0, 3, 1, 2, 2, 0, 1, 2, 0, 1])

# Create a new dataframe we are going to perform inference with
sample_df = pd.DataFrame([[107, 2017, 100, 0, 'BMW']],
                           columns=['max_power', 'year', 'mileage',
                                   'fuel', 'brand'])
sample_df

```

	max_power	year	mileage	fuel	brand
0	107	2017	100	0	BMW

```

# Encode and scale each feature
sample_df = one_hot_transform(brand_ohe, sample_df, 'brand')
sample_df

```

	max_power	year	mileage	fuel	brand_Ashok	brand_Audi	brand_BMW
0	107	2017	100	0	0.0	0.0	1.0

	brand_Chevrolet	brand_Daewoo	brand_Datsun	...	brand_Mitsubishi
0	0.0	0.0	0.0	...	0.0

	brand_Nissan	brand_Opel	brand_Peugeot	brand_Renault	brand_Skoda
0	0.0	0.0	0.0	0.0	0.0

```
brand_Tata  brand_Toyota  brand_Volkswagen  brand_Volvo
0          0.0          0.0          0.0          0.0

[1 rows x 35 columns]

output = model.predict(sample_df)
output

/tmp/ipykernel_22/3691783347.py:120: RuntimeWarning: overflow
encountered in exp
/tmp/ipykernel_22/3691783347.py:120: RuntimeWarning: invalid value
encountered in divide

array([2])
```

Report

A3: Car Price Classification

1. Introduction

This project focuses on developing a custom classification model to categorize car prices into four categories. The main goal was to evaluate different regularization techniques, optimization methods, and hyperparameters to identify the best-performing model. The model was trained using batch, mini-batch, and stochastic gradient descent methods, with both normal and ridge regularization.

2. Experimental Setup

2.1 Dataset

- The dataset contains labeled car price data divided into four categories: 0, 1, 2, 3.
- The model aims to classify each car accurately into one of these price categories.

2.2 Model Configuration

- **Regularization Techniques:**
 - Normal
 - Ridge
- **Optimization Methods:**
 - Batch

- Mini-batch
 - Stochastic (stochastic)
- **Learning Rates (α):** 0.01, 0.001, 0.0001

The training process involved iterating over all combinations of regularization, optimization methods, and learning rates. The model with the lowest training loss was selected as the best model.

3. Evaluation Metrics

The following metrics were computed for each class:

- **Precision:** Proportion of correctly predicted samples to total predicted samples in that category.
 - **Recall:** Proportion of correctly predicted samples to all actual samples in that category.
 - **F1-Score:** Harmonic mean of precision and recall.
 - **Accuracy:** Overall proportion of correct predictions.
 - **Macro Average:** Average metric across all categories equally.
 - **Weighted Average:** Average metric weighted by the number of samples in each category.
-

4. Results

4.1 Classification Performance

Category	Precision	Recall	F1-Score
0	0.744	0.883	0.808
1	0.657	0.575	0.613
2	0.557	0.389	0.458
3	0.717	0.876	0.788

Overall Metrics:

Metric	Value
Accuracy	0.686
Macro Avg	0.667
Weighted Avg	0.672

Observations:

- Categories 0 and 3 show high precision and recall, indicating the model predicts low-priced and high-priced cars well.

- Category 2 performs poorly, suggesting mid-range car prices are more difficult to predict or have fewer representative samples.
 - Overall, the model achieves ~68.5% accuracy.
-

4.2 Best Model Selection

The best model was determined by evaluating final training loss across different combinations of regularization, optimization methods, and learning rates.

Best Model Configuration:

- **Regularization:** Normal
- **Optimization Method:** Stochastic (sto)
- **Final Training Loss:** 0.2487

Conclusion:

The stochastic gradient descent method with normal regularization yielded the best performance in terms of training loss. This model balances learning efficiency and generalization capability across the dataset.

5. Conclusion

- The custom classification model successfully categorized cars into four price categories with reasonable accuracy.
- Performance varies across categories, highlighting the need for further improvements, especially for mid-range cars.
- Stochastic optimization with normal regularization proved most effective among the tested configurations.
- Future work could include:
 - Balancing category distributions.
 - Experimenting with advanced regularization techniques.
 - Further tuning of hyperparameters.